ABRA — the plain-English deck

Version 5.265.0 · 2026-09-06 · Will Hooper

5.265.0 - A MOVE THAT CLEANS UP THE FIELD WAS DOING THE RIGHT THINGS IN THE WRONG ORDER, AND NO TEST WE HAD COULD SEE IT.

THE TWO QUESTIONS THIS VERSION ANSWERS. *Have we tested Rapid Spin clearing hazards and things like Leech Seed?* Partly. The move does four things — it sweeps your own side of the field, it pulls the Leech Seed off you, it frees you from a wrap, and it gives you a Speed boost. We had a real test for the first, a second-hand one for the Leech Seed, and **nothing at all** for the wrap: the simulator kept a counter for it that no test ever looked at. *Have we tested that Leech Seed actually heals the one who planted it?* Yes, properly — two tests read the planter's health with the planter deliberately hurt first, so the test could not pass by accident on a Pokemon that was already full.

WHAT THE SECOND QUESTION TURNED UP. Three different moves clean up the field, and the real game writes down what they did in three different orders. We used one order for all three. **The end position is identical either way**, which is why every board test we had passed over it for as long as it existed. It is a commentary difference, not a board difference — and commentary is its own bar here, not something to wave through. It is fixed to match the real game, driven off information we already had rather than by naming the moves in the code.

WHAT WE CAN AND CANNOT REPORT. Our list of tested mechanics is now **830 of 830, with none missing**. But changing the simulator means every big measurement taken before the change was taken on different code, so **those numbers are withheld rather than repeated with a note attached** — a note is not a quarantine, and this project has paid for that twice. Nine health checks run and seven are failing — **every one of them because the measurement is older than the change, not because anything was caught doing the wrong thing.** That is worse than a failure we can point at, not better, and it is why the honest answer this version is a blank rather than a repeated number. Re-running them is the next job and the commands are written down.

AND WE SAID IN ADVANCE THAT THE LADDER SCORE WOULD NOT MOVE, AND IT DID NOT. Rapid Spin does not appear once in the frozen set of real ladder teams we measure against, so a fix to it cannot show up there. We ran the same games on the old code and the new code and got the same result on both sides, down to identical logs. That is the split working as intended: the lab catches the rare thing, the ladder sample tells us whether it matters.

5.264.0 - ONE CHANGE, AND IT IS TO OUR MEASURING RIG RATHER THAN TO THE GAME. THE "DIFFERENT BOARD" SCORE FELL FROM 34 GAMES OUT OF 961 TO 27; THE "DIFFERENT COMMENTARY" SCORE FELL FROM 100 TO 93.

WHAT WAS WRONG, IN PLAIN TERMS. When we check our simulator against the real one, we make both of them draw from the same list of random numbers, so that any disagreement has to be a rule and cannot be luck. Each draw has an address — this turn, this move, this target, the first draw, the second draw. The real simulator turns out to roll a die for a certain family of moves and then never look at the result: it is a leftover, and the value is thrown away every single time in this format. We did not roll that die, because there is nothing to roll it for. But the real one did, and it used up an address doing it. So every later draw in the same moment sat at a different address on their side than on ours, and the two engines quietly rolled DIFFERENT numbers for the same event — a burn, a poison, an ability going off. Seven games were decided by that and not by any rule.

THE FIX IS TO GIVE THE POINTLESS DIE ITS OWN DRAWER. It still gets rolled, exactly as the real game rolls it, so nothing about their behaviour changes. It simply stops shifting everything behind it along by one. No line of the simulator itself was touched.

WE CHECKED FIVE SUSPECTED CASES AND ONLY THREE WERE THIS. Poison Touch, Flame Body and Cursed Body were all confirmed, each against a control set up on the same Pokemon that we knew should agree. The sleep case was NOT this — we staged it, both engines put the target to sleep for the same three turns, and the disagreement is something else that is still open. The freeze case was predicted not to move and did not. We wrote both of those predictions down before running anything, which is the only reason we can call them refutations now instead of quietly dropping them.

THE PART WE ARE MOST CAREFUL ABOUT IS HOW WE ARE ALLOWED TO CLAIM THE IMPROVEMENT. Changing the measuring rig means the old measurement and the new one are not strictly the same experiment, and we have a tool whose whole job is to say so. It said NOT COMPARABLE, and named both reasons. So we did not publish the improvement off that pair at all. We ran a THIRD version instead: the new code, with a switch flipped that puts the old behaviour back. The tool calls that pair comparable, and it reproduced the old result to every last digit — the same board score, the same commentary score, the same number of turns compared, the same number of games we could not use. That is what lets us say the improvement is due to this one change and nothing else. One honest caveat: the tool cannot see a switch flipped in the environment, so both files write down which way the switch was set, and a reader who trusted the tool alone here would be trusting a check that is blind to the one thing that moved.

TEN OF OUR TWELVE PREDICTIONS LANDED, AND WE NAME THE TWO THAT DID NOT. We called the board score at 30 and it came in at 27, inside the range we had written down. And we claimed the new drawer would only ever catch that one kind of die; on real games it also catches a duration roll for Outrage. That one is harmless, and the check that would have quietly passed was tightened rather than left alone.

WHAT IS STILL BROKEN, NAMED. Five games have boards that drift apart while nothing is ever said differently — the hardest kind, because there is no message to search for. Three of those five are one thing: a hidden counter for how many times a Pokemon has protected in a row. We tried the obvious setup to reproduce it and it did NOT reproduce, so the simple explanation is already handled and the real condition is something else. Saying that is worth more than a guess we would have to take back.

AND NOTHING WE ARE WAITING ON BECAME PUBLISHABLE. Our headline number - how well the engine's own judgement predicts a winner - is still withheld, not printed with a warning label next to it. It becomes available when the engine gate opens and the measurement is run again, and not before.

5.263.0 - FIVE MORE FIXES. THE "DIFFERENT BOARD" SCORE FELL FROM 41 GAMES OUT OF 961 TO 34; THE "DIFFERENT COMMENTARY" SCORE FELL FROM 108 TO 100.

WHAT THEY WERE, IN PLAIN TERMS. In the real game, when the last Pokemon on the other side faints, a move that would raise its user's own stats simply does not get to. Ours raised them anyway. In the real game, a Pokemon that attacks and then swaps out does not swap out if that attack ended the battle. Ours kept swapping. Some Pokemon have an ability that makes a burn hurt them half as much; we knew about abilities that ignore a status entirely and abilities that turn one into healing, but not about abilities that merely turn one down, so the burn was doing full damage. A Pokemon that mega evolves does so after everything else on the board has settled, which means an item that fixes a lowered stat gets eaten first; we ran the mega too early. And a Pokemon with a pickpocketing ability cannot steal an item after it has fainted; ours was robbing the attacker from beyond the grave.

THE LAST ONE ONLY BECAME VISIBLE BECAUSE OF THE ONE BEFORE IT. We fixed the timing of the pickpocket last session, which moved the theft to a moment when the thief might already be dead. Nothing was broken by that fix; something that had always been wrong simply stopped being hidden behind it. That is worth saying because it is the opposite of the usual story: the count went the right way both times, and the second problem was created by nobody.

THE BIGGEST THING WE LEARNED IS NOT A FIX. For three rounds of this work we have been picking what to fix from a table. The table sorts games by the first place the two engines said something different. The score we are actually judged on sorts games by the first place the two engines had a different board. Those are two different lists, and for three rounds we were reading the first one and moving the second one. Re-reading against the right list changed what is worth doing next: a quarter of what is left is our measuring rig rather than the game, twelve of the remaining thirty-four are one single problem wearing five names, and five of them do not appear on the old table at all because nothing was ever said differently in those games — the boards just quietly drifted apart.

A NUMBER WENT UP LAST TIME AND THIS TIME IT WENT DOWN. Games where only the commentary differs fell from 71 to 70. Added to the board column that is 112 games before and 104 now. We keep reporting the two together because fixing a board does not delete a game, it moves it between columns, and a column read alone can be made to say anything.

WE PREDICTED TEN NUMBERS AND GOT NINE. The one we missed we had already explained in writing before the run: we said one game would move between columns and one would get re-filed under a different cause, and it did — and the thing it got re-filed under turned out to be the fifth fix. A prediction that misses for a reason you wrote down first is more use than one that hits.

5.262.0 - SIX MORE FIXES. THE "DIFFERENT BOARD" SCORE FELL FROM 50 GAMES OUT OF 961 TO 41; THE "DIFFERENT COMMENTARY" SCORE FELL FROM 114 TO 108.

WHAT THEY WERE, IN PLAIN TERMS. When a Pokemon punishes an attacker for touching its shield, the real game lowers the attacker's Attack through the same door every other stat drop goes through — so an ability that inverts drops, an ability that hits back at whoever lowered a stat, and an ability that simply refuses to be lowered all get their say. Ours wrote the drop straight in, and all three abilities sat there doing nothing. When a Pokemon is handed a new ability part way through a battle, the real game starts that ability up; ours only shut the old one down, so a swapped sand-summoner summoned no sand. The real game checks twice during a move whether anybody needs to react — after the hit and again after the recoil — and we only checked once, which is why a berry kept being eaten at the wrong moment. A thieving ability was being paid too early. A Pokemon that pays its own health to escape left before the board had settled. And a move that gets pulled towards a different target kept trying to split itself between two.

WE WROTE DOWN WHAT WE EXPECTED BEFORE EACH RUN AND THEN RAN IT. Every fix has a switch that puts the old behaviour back, and a small test that was shown failing before a single line of the simulator was changed. One of the six had its reasoning corrected by its own test before any code existed: we thought only some kinds of redirection cleared the flag, the test read the real game's source on every run, and it said plainly that all of them do.

A NUMBER WENT UP AND IT IS NOT A REGRESSION. We now count two things separately: games where the two engines end up with a different board, and games where they only describe things differently. That second count went from 69 to 71. Fixing a board does not delete a game from the tally — it moves it from the first column into the second. Added together the two columns fell, from 115 games to 112. Saying this out loud is the point: a number that rises for a good reason looks exactly like a number that rises for a bad one, and only the explanation tells them apart.

THE MEASURING TOOL HAD DISQUALIFIED ITSELF. The tool that decides which of our numbers are still trustworthy scans code for a reference to the simulator. Its own self-test contains a fake line of code with exactly that reference in it, written as text. So the tool decided it was part of the simulator, refused to trust itself, and took seven other reporting tools down with it. That is fixed, and the rule is now worked out rather than kept as a list: anything the gate has to read in order to decide is, by definition, something it is allowed to quote. Nine reports came back; nothing that depends on the simulator did.

AND THE HEADLINE NUMBER IS STILL NOT PUBLISHED. Whether the model's win estimate is honest — the one number this whole project is judged on — is still not printed anywhere in this deck, because it is measured through a simulator that is not yet correct. It comes back when the gate opens, not when it feels ready.

5.260.0 - FOUR MORE FIXES. THE "DIFFERENT BOARD" SCORE STAYED AT 50 GAMES OUT OF 961; THE "DIFFERENT COMMENTARY" SCORE FELL FROM 151 TO 114. All four were about the simulator saying the wrong thing, or saying nothing, rather than about it doing the wrong thing — which is why the board score did not move and we said in advance that it would not.

WHAT THEY WERE, IN PLAIN TERMS. When a Pokemon has run out of every move it knows, it flails — and the real game announces that before it happens. Ours did not. When you use the move that reads the opponent's held item, the real game only names the item if the move actually connects; ours named it even when the move was blocked, missed, or hit something immune. A Pokemon recovering from a huge attack cannot act — and the real game applies that BEFORE it applies sleep, so a sleeping Pokemon that owes a recovery turn does not tick its sleep counter down. Ours ticked it, which meant it woke up a turn early. And finally: if everything a move could have hit has already fainted this turn, the real game prints "But it failed!" and ours printed nothing at all. **That last one was the single biggest gap left in the file, 32 games, and nobody had spotted it — because we had been reading a sample of forty games instead of the full list.**

WE WROTE DOWN WHAT WE EXPECTED BEFORE EACH RUN, AND WE MISSED THREE TIMES OUT OF FOUR — BY ONE, ONE AND TWO GAMES. Every miss was the same mistake in the same direction: counting how many games mention a problem tells you the most that can be fixed, not how many will be. Some of those games have a second, unrelated problem sitting behind the first one, so repairing the first just reveals the second. That is now measured rather than guessed at: about nine in ten close outright.

ONE THING WORTH SAYING BECAUSE IT IS THE OPPOSITE OF OUR USUAL MISTAKE. Four times this session, a new test accused the simulator of a fault the actual measurement does not count — because the test was fussier than the thing it was defending. One of them accused its own control group. We usually pay for tests that are too lenient; these were too strict, and a too-strict test invents work.

5.259.0 - WE CORRECTED TWO NUMBERS WE HAD PUBLISHED, AND IN BOTH CASES THE PROBLEM WAS WHERE WE SAID THEY CAME FROM. The first said our "did they bring all four" filter keeps games a certain amount longer, and pointed at a file that has never contained that figure — not today, and not in any of the six versions it has ever had. We replaced it with the step that file really does record, and we deleted the ratio outright rather than guess at a new one. The second was a score for how well the engine's judgement predicts a winner. That score was faithfully copied from a real file, but the file is QUARANTINED because it was measured against a simulator we have since rewritten, so the honest thing is to print nothing at all. A warning label next to a number people still read is not a warning; it is a number.

5.258.0 - THE NUMBER WE PUBLISH AND THE NUMBER WE MEASURED ARE THE SAME NUMBER AGAIN. IT IS 50 GAMES OUT OF 961. For several nights running, this page has had to say that the score everyone reads was deliberately not being rewritten, because the engine and the measuring tool were both being edited and rewriting a score in the middle of that is how a number appears that nobody can trace. Tonight the tree was settled, so it was rewritten. `data/game-differential.json` **now holds 50 games**

out of 961 where our simulator and the real one end up with a different board, and 151 out of 961 where their running commentary stops matching. The file had been carrying 46 from an older engine for a day and a bit, and while it did, both of the questions the project uses it to answer were failing for the wrong reason - they were failing because the file was out of date, not because of anything measured. **Now they fail on real counts, which is worse-sounding and much more useful.**

FIVE THINGS LANDED TONIGHT AND EACH ONE WAS MEASURED ON ITS OWN, SO WE CAN SAY WHICH ONE DID WHAT. We started the night at 59 different-board games. Big Root took it to 58. Leech Seed took it to 56. A repair to the measuring tool itself moved nothing at all - which was the prediction, and it was checked rather than assumed. Fairy Aura took it to 51. Beat Up took it to 50. Every one of them had a test that was shown FAILING before the fix and passing after it, plus a switch that puts the bug back and proves the test can tell the difference.

BIG ROOT IS AN ITEM THAT BOOSTS HEALING, AND IT NAMES FIVE KINDS OF HEALING. WE HAD IMPLEMENTED ONE. The real game's item boosts draining attacks, Leech Seed, Ingrain, Aqua Ring and Strength Sap. Our engine only ever looked at draining attacks; the other four went unboosted. There is a second half that is easy to get wrong even once you know the list: the game rounds the healing DOWN first and applies the boost second, so a Pokemon healing a sixteenth of 155 health heals 9 and then 12 - not 11, which is what you get if you round at the end. We found it in a real stored game rather than in the lab: the real game healed a Meganium to 71 and we healed it to 68.

LEECH SEED WAS STILL DRAINING A POKEMON AFTER THE POKEMON DOING THE DRAINING WAS DEAD. In the real game, the seed checks each turn whether whoever planted it is still on the field and alive, and **if it is not, the seed does nothing at all that turn.** Our engine made that check too - but only to decide whether anybody got healed. It went on taking a chunk of health off the seeded Pokemon regardless. That is not a cosmetic difference; it is health disappearing from a board that the real game leaves alone.

THE MOST USEFUL FINDING OF THE NIGHT IS ABOUT THE MEASURING TOOL, NOT THE GAME: A SETTING THAT WAS SUPPOSED TO POINT AT ONE TEST CONFIGURATION HAD BEEN POINTING AT A DIFFERENT ONE SINCE 13 AUGUST. Our comparison harness has several "configurations" - a maximum-damage one, a minimum-damage one, a realistic-dice one. Three fixed test setups were written to use the maximum-damage configuration and were wired up by position: "use the first one in the list." On 13 August somebody added a new configuration to the front of that list, and the three test setups silently started using it instead. **The four failing checks that this caused were all one line, and the engine was never at fault.** We proved that rather than asserting it: run the same measurement fourteen times and our numbers never move, while the reference implementation's numbers wander around and once lost their own minimum. They are now wired up by NAME, with an error thrown if the name ever disappears. **Two of those checks had been passing purely by luck for fourteen days.**

FIXING THE MEASURING TOOL MEANS THE BEFORE AND AFTER ARE NO LONGER STRICTLY COMPARABLE, AND WE PAID FOR THAT RATHER THAN ARGUING ABOUT IT. One version ago we added a fingerprint of the measuring tool's own code for exactly this reason. The fingerprint changed, which is the fingerprint doing its job, so the measurement was simply re-run on both sides with everything else held identical. Every count came back the same to the last digit.

A CORRECTION, AND IT IS THE MOST IMPORTANT THING ON THIS PAGE. THE "1.53 TIMES TOO MUCH PROTECT" WE PUBLISHED LAST NIGHT WAS MEASURED AGAINST THE WRONG YARDSTICK. We said our stand-in player presses Protect about half again as often as its own instruction table says, and blamed the way the table is squeezed onto the four moves a Pokemon actually carries. **The squeeze is real and it is about half the story. The other half is the yardstick.** Broken down over the same 17,532 decisions: the instruction table across the whole ladder says 13.565%; the same

table, weighted by the Pokemon this test actually plays, says **16.209%** - because the test deliberately plays a scenario-seeking mix of teams, not a random slice of the ladder; the squeeze then takes it to 20.257%; and the stand-in actually pressed it 20.374%, which means **the stand-in itself is faithful to within half a percent of what it was told.** So the honest statement is 1.257 times, not 1.53, and any sentence of the form "it reads X% against 13.565%" needs the matching figure for the actual pool of teams beside it.

AND ONE PART OF LAST NIGHT'S EXPLANATION IS WITHDRAWN OUTRIGHT: WE BLAMED POKEMON HAVING FEWER THAN FOUR LEGAL MOVES, AND THE EFFECT RUNS THE OTHER WAY. 87.0% of decisions have all four moves available, and those are precisely the ones pressing Protect most - 21.724%. A Pokemon down to one legal move is usually down to its attacking move, not its Protect, and those decisions press Protect **less**. Measured properly against real human play, and held out so it cannot be fitted to its own answer, the squeeze over-predicts human Protect clicking by about 1.11 times, not 1.53 times, and we know that is real rather than noise because we measured the noise: splitting the human games in half and comparing the two halves gives a spread of **0.002 percentage points**, and the over-prediction is +1.644 and +1.646 points.

TWO OF OUR FIVE PREDICTIONS FOR TONIGHT WERE WRONG, AND THEY WERE WRONG THE SAME WAY. Before each fix we wrote down what we expected the score to become. Three landed exactly. The Leech Seed one said 58 and got 56; the Fairy Aura one said 54 and got 51. Both times the mistake was reading a list of example failures **that is capped at 40 entries** as though it were the whole list. It is a sample. Both misses were in our favour, which is not a defence - a prediction that is only checked when it flatters you is not a prediction.

WHAT IS STILL NOT FIXED, SAID PLAINLY. The overall health check reads **7 of its 9 questions passing**, and the two failures are the two whole-game ones. Nothing that was locked away becomes readable again: how well the engine's position estimate is calibrated, every rollout result and every head-to-head all stay withheld rather than printed with a warning label. The big model refit is still owed and is still a refit rather than a re-stamp - no weights were touched. Three known engine gaps are named and not fixed: a Struggle announcement, a Poltergeist announcement made at the wrong moment, and a recharge turn given too high a priority. Two more announcements are missing entirely - Fairy Aura and Unnerve never announce themselves on entry - and both are measured and filed rather than claimed done. About 6 remaining mismatches, and 5 more of a different shape, are in the measuring tool rather than in the game, and we say which and why instead of counting them against the engine.

5.257.0 - OUR STAND-IN PLAYER WAS NOT PICKING "PROTECT" TOO OFTEN. IT WAS BEING HANDED A MENU WITH ONE ITEM ON IT. To test our simulator we replay stored games with a stand-in player making the clicks. That stand-in was pressing Protect roughly twice as often as a human does, and the obvious reading was that it liked the move too much. That reading was wrong. A setting meant to nudge it towards certain moves was instead deleting all the others: **on 22.2% of its decisions it arrived at the choice with exactly one option left, and 60% of the time that one option was Protect.** More than half of its Protect clicks were never a choice at all. Where it did have all four of its moves in front of it, it was already pressing Protect **15.3%** of the time, which is the human rate. Nothing was wrong with how it chose; something was wrong with what it was offered.

FIXING IT MADE THE TEST GAMES ACTUALLY FINISH, AND THAT IS THE PART THAT MATTERS. Protect is the move that makes a turn happen with nothing in it, so a player pressing it constantly produces games that grind to the time limit and stop. **The share of test games that reach a real ending went from 56% to 79%** - 539 games out of 961 before, 762 after. The number of games where our simulator and the real one end up with a different board went **from 34 to 47 on the same pair of**

runs, and that increase is what a fix looks like here, not a regression. Games that actually finish reach late positions the old stand-in never got to, so there is simply more game to disagree about. We say that out loud because a score going up looks like bad news and this one is not.

WE ALSO FINISHED THE LIST OF THINGS THE COMPARISON WAS BLIND TO: IT NOW CHECKS 54 KINDS OF HIDDEN STATE INSTEAD OF 40, AND NOTHING IS LEFT ON THE LIST. Two of the sixteen could not be done, and in both cases we can show WHY rather than assert it: one of them is not a thing our engine stores at all - it recalculates the effect on the spot - and the other is a move that this format allows but that no legal Pokemon in it can learn. The check that counts them now works that out on every run, so if a Pokemon ever does learn it, the check goes red by itself instead of us remembering.

A CORRECTION, AND IT IS THE MOST IMPORTANT ITEM ON THIS PAGE. WE SAID ONE OF OUR MEASURING SETUPS WAS GIVING DIFFERENT ANSWERS TO THE SAME QUESTION. IT WAS NOT. Earlier tonight we published that three identical runs came back with 167, 167 and 138, that we did not know why, and that until we did, none of that setup's numbers could be quoted. **That is withdrawn.** Six runs on identical settings split perfectly cleanly into two groups either side of the moment the Protect fix landed at 02:27: three before it, three after it, and each group is identical to itself down to the last internal counter. There was never any randomness to explain. The number in question - 53 - stands, and it repeats.

THE REAL PROBLEM UNDERNEATH IS BIGGER THAN THE ONE WE WITHDREW: WE FREEZE EVERYTHING THE TEST READS AND WE NEVER FROZE THE TEST ITSELF. Every measurement here is taken with the simulator frozen, the list of scenarios frozen and the pool of teams frozen. All three of those are things the measuring tool READS. The measuring tool itself was never fingerprinted, so two runs of "the same test" could be two different tests and nothing would say so. Worse: we have a program whose entire job is to answer "are these two runs comparable?", and asked about two of these runs it answered **"COMPARABLE - a difference between their numbers is the change under test"** - about two runs made by different code. Its own notes already described this exact gap, in writing, in the file. **A hole that is written down is not a hole that is closed**, and that is the lesson worth keeping. The measuring tool now fingerprints itself, refuses to publish a run if it was edited while that run was playing, and works out that warning from the two runs in front of it rather than repeating a sentence somebody typed.

A SECOND CORRECTION: ONE FIGURE WE PUBLISHED TONIGHT SHOULD READ 55, NOT 47. Both are real and neither was a mistake in measuring - 47 was measured before we widened the comparison from 40 checks to 54, and 55 is the same run through the wider comparison. It is the same reason a temperature is meaningless without saying which thermometer: we now name the run and the setup beside every one of these scores, every time.

AND THE STAND-IN STILL PRESSES PROTECT HALF AGAIN AS OFTEN AS ITS OWN INSTRUCTIONS SAY, WHICH IS A DIFFERENT FAULT AND WE ARE NAMING IT RATHER THAN QUIETLY DIALLING IT OUT. The table it reads gives how often each Pokemon uses each move across the whole population; the stand-in then squeezes that down onto the four moves the Pokemon in front of it actually has. Protect survives that squeeze almost every time, so it gets inflated - about **1.53** times its own input. We have not fixed it in the same pass, on purpose: it changes what the stand-in is being told, and fixing two things at once means neither can be credited.

THE SCORE WE PUBLISH IS STILL A DIFFERENT NUMBER, ON PURPOSE. The file everyone reads, `data/game-differential.json`, was not rewritten and still says 46 games out of 961. It is stale as a description of tonight's simulator, and we leave it stale rather than overwrite it while the engine and the measuring tool are both being edited - that is exactly how a number appears that nobody can trace back. Everything above lives in the working files under `data/verification/`, each with a deliberately-broken control run beside it to prove the change is what moved it.

5.256.0 - A MOVE THAT SPENDS ONE TURN CHARGING AND LANDS ON THE NEXT WAS COMING BACK OUT AND HITTING THE WRONG POKEMON - AND NOTHING WE OWN COULD HAVE SEEN IT UNTIL TONIGHT. Some moves take two turns: the attacker disappears or winds up, and the hit arrives a turn later. The real game remembers which of the two enemies you aimed at when you started, and lands the hit there. Our simulator did not remember; it simply hit whichever enemy was on the left. So a move charged at the right-hand Pokemon came out and struck the left-hand one. **The reason nobody had ever caught this is the interesting half.** Until tonight, the stand-in player we use to drive these test games always aimed at the left-hand enemy anyway, with both of its Pokemon. If the aim never changes, then remembering the aim and forgetting it produce exactly the same game, and no comparison in the world can tell them apart. **A flaw in our stand-in player had been hiding a real flaw in the game engine.** The problem we had written on the card was real too, and turned out to be the smaller of the two: the charging move surviving a refusal it should not survive happens 2 times in 961 games.

THE SCORES MOVED A LOT, AND EVERY ONE OF THEM BELONGS TO A NAMED STAND-IN PLAYER. We replay 961 stored games through both simulators and count the ones where the two end up with a different board. There are two different stand-in players driving those games, they play differently, and the program deliberately refuses to compare a score from one with a score from the other. **With the new stand-in player, the different-board count went from 110 to 53.** With the older one it went from 35 to 34. The count of games we had to throw away because the two simulators never rolled a die at the same place fell from 38 to 4, and charging moves have vanished from that list entirely. Everything we can probe is still probed - 829 of 829 - and the engine gate is back to failing 2 of its 9 questions.

THE PUBLISHED NUMBER IS STILL A DIFFERENT NUMBER, AND THAT IS ON PURPOSE. The file we publish, `data/game-differential.json`, was again not rewritten and still reads 46 of 961. That is what the status tool prints, and it is stale as a description of tonight's engine. The new measurements sit in this version's working files instead. Two numbers exist for one question by design, and neither may be quoted without saying which file it came from.

WE DID NOT ASK ANYONE TO TAKE THE 57-GAME IMPROVEMENT ON TRUST. We re-ran the whole thing with the same code, the same games and the same stand-in player, with only the two fixes switched back off. The old numbers came back exactly - 110 and 35, against 53 and 34 with the fixes on. And a separate tool confirms that only one file changed between the two runs: the simulator itself. **So the whole 57-game improvement is these two fixes and nothing else.** That is a much stronger claim than a before-and-after taken on a tree that was also being edited underneath it.

THE TEST SETUP THAT CAUGHT THIS WAS ONE HOUR OLD. Earlier the same night, this exact charging problem was written down as real but impossible to stage: our scripted test could not even make the simulator play the second half of a two-turn move, because it was filling in a piece of information the game does not give you for a move you are locked into. So no staged test in this entire project had ever played out a charging move's landing turn. The new stand-in player's thrown-away games are what showed us how to stage it, and the repair was one line. **Every staged test can now reach a charging move's second turn, and none of them ever could before** - and nothing else in the staged suite moved, which we checked rather than assumed.

THIS IS THE SECOND TIME IN ONE NIGHT THAT CHANGING THE STAND-IN PLAYER REVEALED A PROBLEM RATHER THAN CREATING ONE. Earlier we swapped in a stand-in that plays games which actually finish, and the different-board count went from 0 to 135 - not because anything broke, but because we had been scoring games that never ended. Tonight we swapped in one that aims at both enemies, and it uncovered a targeting flaw that a fixed aim had made impossible to observe. **Neither number was wrong. Each was a statement about which games were being played.**

AND A TEST THAT HAD BEEN PASSING ALL ALONG TURNS OUT TO HAVE BEEN PASSING BY LUCK. One of our checks was written to cover a set of settings, and its filter accidentally scooped in a setting whose value is re-rolled at random every time. It only went green when that roll landed under 0.01. It was not catching anything; it was winning a coin flip. Fixed, and now every setting really is covered.

THE HONEST COLUMN: WE PREDICTED 8 THINGS BEFORE THE RUN AND GOT 3 OF THEM RIGHT. On the older stand-in player both misses were off by one. On the new one all three misses went the same way - we called the improvement much smaller than it turned out to be. That is a bias in our guessing rather than noise in the measuring, and it is only visible because the misses get written down beside the hits.

WHAT WE STILL WILL NOT QUOTE, AND WHAT IS STILL OWED. Everything downstream of the simulator stays withheld: how well the model predicts a win, every rollout figure, every head-to-head. The model's weights still need a full re-fit rather than a re-stamp. The engine's own page still lists the charging problem as open and was not restamped in this pass. One of the games that changed is attributed but not yet explained. Thirteen of the 53 remaining games are unnamed, because the file that names them stops at 40. And one half of the second fix is wired up but has not been staged in a real game yet.

5.255.0 - AN INVISIBLE CHANGE TO A FILE - NOT ONE LETTER OF IT DIFFERENT - THREW AWAY FIVE LONG MEASUREMENT RUNS BEFORE ANYONE WORKED OUT WHY. THE SIMULATOR NOW SAYS WHICH KIND OF CHANGE IT WAS. Every measurement here is taken against a frozen photograph of the code, and that photograph has a fingerprint. If the fingerprint moves, the measurement is stale and has to be taken again. Tonight the fingerprint moved and nothing had actually changed: one file had been re-saved with Windows-style line ends instead of Unix ones, so every line looked new to the fingerprint while all 26 frozen files were letter-for-letter identical. An agent found the engine gate failing on 7 of its 9 questions instead of the usual 2, and spent five heavy re-runs putting it back. The same case is sitting in the tree right now - one file, 39,932 lines one way against 39,932 lines the other way, zero characters edited.

WE DID NOT MAKE THE PROBLEM GO AWAY, WE MADE IT SAY WHAT IT IS - AND THAT DISTINCTION IS THE WHOLE POINT. Nothing is forgiven. The fingerprint still counts raw bytes, it still moves when the line ends change, and every measurement it stales is still stale. What is new is that the tool now tells you WHICH of the two happened: the files really changed, or the files are identical apart from how their lines end. Tonight's two failing questions are labelled "the files really changed" and they still fail, with every count held back and the words "a re-measurement IS owed" printed where a number would be. **The two obvious ways to make this stop are both correctly closed off.** Forcing those nine files to one style would rewrite them, move every frozen photograph's fingerprint, and break a test suite that deliberately looks for the Windows line ending in the simulator's own text. And teaching the comparison to ignore the difference is banned here, because the difference is real and something else can already see it. So this is a third door: a diagnosis, not an excuse.

THE TOOL FOUND A BUG IN ITSELF BEFORE WE TRUSTED IT, WHICH IS EXACTLY WHAT SHOULD HAPPEN TO A THING BUILT TO STOP FALSE ALARMS. Its first version reported two identical files as different - a false alarm from the false-alarm detector. It was then deliberately broken in two different ways to prove it could fail, scoring 9 of 16 with the fix disabled and 10 of 16 with it over-applied, before scoring 17 of 17 repaired.

THE SCORE MOVED AGAIN, AND TWO REAL THINGS IN THE GAME ARE NOW RIGHT. We replay 961 stored games through both simulators. The games where the two end up with a different board went from 37 to 35, and the games where they first say something different went from 122 to 120. Both games

that closed were traced by name, and no new one opened. **The published file, `data/game-differential.json`, was again deliberately not rewritten and still reads 46 of 961** - stale as a description of tonight's engine. Say which file a score came from, every time.

THE TWO FIXES, IN PLAIN TERMS. Imprison is a move that seals away any move the opponent shares with you. Our simulator put up the flag and sealed nothing, so the opponent went right on using a sealed move, doing damage with it and spending its power points - a rule that looked implemented from the outside and did nothing at all. And Magic Bounce, the ability that reflects a move back at whoever used it, was reflecting the wrong half of Parting Shot: the move lowers stats and then sends its user to the bench, and we were lowering the wrong Pokemon's stats and sending the wrong Pokemon to the bench. Both were tested by staging the real situation, and both came with controls that stayed still while the fix moved.

WE ALSO WIDENED WHAT THE COMPARISON LOOKS AT, ON WILL'S CALL, BECAUSE A COMPARISON THAT DOES NOT LOOK AT SOMETHING AGREES ABOUT IT FOR FREE. There are 56 pieces of hidden state a Pokemon can be carrying. Our comparison only checked 37 of them, so a perfect score was a smaller claim than it sounded. Three more are now checked, taking it to 40. Each one was traced to a place the simulator really writes it AND a place it really reads it before being wired up - a lesson from a previous round where something looked wireable everywhere and the engine held nothing under that name. **The game score did not move, and we said so in writing before the run**: the frozen pool of real games contains no Lock-On and no Dragapult, so the laboratory moved and the pool correctly sat still.

AND WE KILLED AN IDEA THAT SOUNDED BETTER THAN IT IS. The proposal was to drive the simulator with real recorded human clicks instead of a model. It cannot work here, and the reason is structural rather than a matter of tuning. A replay stops being a replay the moment a Pokemon faints at a different time than it did in the real game, and at least 24.8% of games in our frozen pool have a damage-decided faint on turn 1 - because Champions team sheets never publish the 66 points a player spends on stats, so our damage cannot match the real game's. On top of that, the comparison deliberately pairs one game's team against a different game's team, so there is no recorded human sequence for the matchups it actually plays.

AND THE NUMBER THAT SETTLES IT WAS ALREADY IN THIS PROJECT AND NOBODY HAD ACTED ON IT. We measured a long time ago that real humans aim both their Pokemon at the same target 23.4% of the time, against roughly half the time if the two choices were made independently. The empirical driver states in its own text that it has no model of targeting and no model of switching, and switching is 12.1% of real decisions. That is why its games run a median of 11 turns when real games run 7, and why 49% of them hit the turn limit instead of ending. A measurement that sits in the code and changes nobody's decision is as good as one nobody took.

WHAT WE STILL WILL NOT QUOTE. Everything downstream of the simulator stays withheld - how well the model predicts a win, every rollout figure, every head-to-head. The gate is still shut. The model's weights still need a full re-fit rather than a re-stamp. And the remaining differences no longer form a big group: after setting aside the ones already filed, the largest group worth acting on was two games. It is a long tail now, which is a better problem than it was.

5.254.0 - A FILE THE SIMULATOR ACTUALLY READS HAD BEEN SIX DAYS OUT OF DATE AND WAS STILL RUNNING A BUG THE REST OF THE PROJECT HAD ALREADY FIXED. THE CHECK THAT CATCHES THIS WAS RUNNING, WAS FAILING, NAMED THE EXACT FILE - AND WAS WRITTEN DOWN AS A NOTE INSTEAD OF ACTED ON. We keep one rulebook describing what every move, item and ability does, and we generate a second copy of it for the version of the simulator that runs in a browser. On 2026-08-29 somebody fixed a rule in the first copy and nobody rebuilt the second, so for six days the browser simulator ran the old rule. It was not a cosmetic difference. Parting

Shot is supposed to send its user back to the bench only when it actually lowers the target's stats; the stale copy did not carry that condition at all, so the move sent its user back to the bench every single time. This is the fifth time a generated copy has drifted away from the thing that generates it.

AND THE PART THAT COST US WAS NOT A MISSING CHECK, BECAUSE THE CHECK WAS NOT MISSING. We have a program whose whole job is to ask whether every generated file is still what its generator would write today. It ran. It failed. It printed the names of the two files that disagreed. It took a second and a half. Then it was filed in a session write-up as one of the things that had got BETTER that day, and nobody rebuilt the file. Our own rule reads: a check nobody acts on is not a check. **So we did not write a sixth checking program.** We moved the one we already have to a place it cannot be skipped - it now runs before every commit, and above the shortcut that used to wave a data-only commit straight through, because "regenerate the rulebook, commit the data" is the exact shape of all five drifts. Two programs that decide the same fact is the most expensive recurring mistake in this project, so adding a third was the wrong answer.

THE SCORE DID MOVE, AND FOUR THINGS IN THE GAME ARE NOW RIGHT THAT WERE WRONG. We replay 961 stored games through both simulators. The count of games where the two end up with a different board went from 41 to 37, and the count where they first say something different went from 128 to 122. Both figures live in `data/verification/fix-batch-7.json`. **The published file,** `data/game-differential.json`**, was deliberately not rewritten and still reads 46 of 961** - that figure is now stale as a description of today's engine, and rewriting it is a job for a pass where nobody is editing the simulator. Say which file a score came from every time you quote one.

THE FOUR FIXES, IN PLAIN TERMS. A move that hurts the user's own stats to hit harder was skipping that cost entirely whenever the target was hiding behind a Substitute doll - the real game still charges you. A sleep that a move chooses inside its own code was being announced as though the move had named it, which it does not. And a body standing in bright sunlight could be frozen: the sky in this game carries a rule saying it refuses to freeze anything, and our simulator only ever read the sky's damage rules, never that one. That last one was proved with nine separate arms, including a control that puts the freeze back in BOTH simulators when a Pokemon on the field cancels the weather.

FOUR OF OUR OWN SELF-TESTS COULD NOT FAIL, AND NOTHING SAID SO. A test suite plants deliberate faults in the simulator to prove it can catch them. Four of those plants no longer pointed at anything real - one aimed at a line whose shape had changed, one at code written a different way - so they were quietly doing nothing. It was invisible because every published run of that suite is written without the switch that turns the planting on. **A suite that has never been shown able to fail is not evidence.** All four were re-aimed and then checked one at a time by hand rather than read off the file that had been hiding them.

FIVE SUSPECTS WERE CLEARED BEFORE ANY CODE WAS TOUCHED, WHICH IS CHEAPER THAN A WRONG FIX. The sleep timer on Dire Claw is correct in both simulators, and that refutation is now a permanent arm of the test rather than a sentence in a report nobody re-reads. Stalling moves - the single largest remaining group, five games - are a coin toss coming out differently, not a missing rule. Castform's forme change was not reachable in either shape we staged, and two abilities from the previous round were shown to fire correctly.

ONE MORE THING WE WROTE DOWN BECAUSE IT WOULD HAVE BEEN INVISIBLE. The program that builds the rulebook was running out of memory. It writes its output near the very end, so running out of memory left the old rulebook on disk with its old timestamp - nothing recorded that the run had died, and every later program would have read a stale file that looked freshly made. The memory it needs is now declared in the file itself instead of being something the next person has to remember, and we checked the rebuild by reading the rulebook's own internal timestamp rather than trusting the exit code.

WHAT WE STILL WILL NOT QUOTE. Everything downstream of the simulator stays withheld - how well the model predicts a win, every rollout figure, every head-to-head. The gate is still shut. The model's weights still need a full re-fit rather than a re-stamp. And one job is deliberately left undone and named: the rulebook builder should rebuild the browser copy in the same action that writes the rulebook, but proving that works needs a run that would invalidate the frozen engine tonight's measurements were taken against.

5.253.0 - ONE OF OUR OWN NOTES SAID "THIS IS STILL BROKEN" AND OUR CHECKER READ IT AS "FIXED", BECAUSE OF A PUNCTUATION MARK IN THE MIDDLE OF THE CELL. We keep a table of everything known to be wrong. Each row has a status cell, and the program that decides whether a row is closed looks at the text after the last vertical bar in the row. If somebody writes a bit of code inside the status cell, and that code contains a vertical bar, the cell gets cut in half and the program reads the wrong part. One row began with the words "open - engine DEFECT" and was being counted as closed. **Nothing was played, nothing was measured and no mechanic changed: the score is still 46 games out of 961.** What changed is that a real problem stopped being invisible.

THE HALF NOBODY HAD NAMED TURNED OUT TO BE THE BIGGER ONE. The bar problem hid 8 rows. A second problem, bold text, hid nine more - the reader skips spaces in front of the status word and does not skip the asterisks that make text bold, so a status written in bold does not register at all, with no bar involved anywhere. Eighteen rows were fixed by changing how they are written and nothing else. Our list of open items went from 237 to 222, and the part of it that says something is actively broken went from 50 to 51 - it went UP, because a genuine defect came back into view.

THE NEW CHECK ASKS A QUESTION INSTEAD OF LISTING THE TWO MISTAKES WE ALREADY FOUND. For every row it asks: does the answer come out the same whether you read the cell the way the program reads it, or the way a person reading the finished page would? That way it catches shapes nobody has thought of yet. It was proved by being shown wrong first - seven made-up bad rows, four of them written in ways nobody in this project has ever used, and then the real list as it stood before the repair, where it correctly names 15 rows and fails. It reads the current list of 506 rows in about a fifth of a second and finds nothing.

AND THE PART WE ARE DELIBERATELY NOT CLAIMING. Sixteen of the rows we made readable were NOT re-tested. Each of those rows now says so, in the row itself. Making a note legible is not the same as proving the thing it describes is fixed, and pretending otherwise would be exactly the kind of quiet upgrade this project keeps paying for. There are also 15 cells still cut by a bar in a way that does not change any answer. We report those and we deliberately did not put them behind a number that is only allowed to go down, because a target like that just teaches the next person to argue that their row is the exception. Ninety more rows with bars in them were checked one by one and left alone.

5.252.0 - THE SCORE WE PUBLISH FINALLY MATCHES THE SCORE WE MEASURED: 77 GAMES OUT OF 961 BECOMES 46 (4.8%). For most of the last day we had two numbers for the same thing. The one we kept measuring kept going down, and the one our published check actually printed stayed at 77, because rewriting the published file while somebody is editing the simulator is how you get a number nobody can trace. This time the measurement was run on a frozen copy of the current engine and written straight onto the published file, `data/game-differential.json`. It now reads 46 games out of 961 where the two engines end up with a different board, and 141 out of 961 where they first say something different. **Every place this project has been saying "the published check still reads 77" can stop saying it** - not because we decided to stop, but because the file was rewritten.

THE MORE IMPORTANT CHANGE IS WHAT KIND OF FAILURE IT IS NOW. Our gate has eight questions that can hold it shut. Earlier in the day six of them failed, and five of those six failed for the worst possible reason: the evidence file could not prove which version of the engine it was measured on, so we knew nothing at all. All five were re-measured against one frozen copy of the engine. **Nothing is in that state any**

more. One question still fails, and it fails for the good reason: a real instrument, run on the engine we have today, found real trouble in 46 real games. Going from "we do not know" to "we know, and it is bad" is progress even when the number itself is not pretty.

WE WROTE DOWN WHAT WE EXPECTED BEFORE WE RAN IT, AND WE GOT ALL FOUR EXACTLY RIGHT - 46 games with different boards, 141 first differences, 7 games that could not be scored, 1 that crashed. Our record on these called-in-advance scoreboards this session is 2 of 3, then 4 of 4, then a one-game miss, then 4 of 5, and now 4 of 4. We keep the misses in the list on purpose.

AND THE REASONING BEHIND THAT PREDICTION HAD A HOLE IN IT, WHICH SOMEBODY FOUND BEFORE THE RUN RATHER THAN AFTER. The brief said the only thing that had changed was some bookkeeping counters, which cannot change a game. That was true of three files and wrong about a fourth: our copy of the outside usage table had rolled over from one month to the next and lost a species, from 284 to 283, and that table does feed the games. It was named in advance as the thing that would prove the prediction wrong, and then it was measured and found to change nothing at all - the detailed breakdown of the differences is byte-for-byte the same as before. That is the difference between a prediction that was right and one that was lucky.

ONE THING WE ARE SAYING OUT LOUD BECAUSE IT WILL COST MORE LATER. The frozen copy of the engine that first produced the 46 was made from files that are in no saved version of the project. It has been replaced by one made from saved files, so today nothing is lost - but a measurement taken against files nobody can get back is a measurement nobody can repeat.

WHAT WE STILL WILL NOT QUOTE. Everything that depends on the simulator being right stays withheld: how well the model predicts a win, every rollout result, every head-to-head. The gate is still shut on that one failing question. The model's weights still need a full re-fit rather than a re-stamp, because the damage table underneath them grew from 318 species to 322 - re-stamping would paper over the very evidence that says the re-fit is needed. Nothing was re-fitted and that model stays paused.

5.251.0 - THE SCORE DID NOT MOVE THIS TIME, BECAUSE THIS ROUND WAS NOT ABOUT THE GAME. IT WAS ABOUT OUR OWN LIST OF WHAT IS STILL BROKEN - AND MORE THAN HALF OF THE ITEMS ON IT THAT NOBODY COULD CHECK WERE ALREADY FIXED. We keep a list of every known problem. **43** of the items on it said "this is broken" while nothing we own could test whether it still was. We went through them one at a time. **24** were already fixed. **6** were the same problem written down twice. The list of open items went from **261** to **237**, and the part of it that claims something is broken went from **74** to **50**. This is not a small piece of admin. Every decision about what to work on next was being made against a list where the untestable half was mostly wrong, and a list nobody can contradict always flatters the plan.

WHICH FILE HOLDS WHICH NUMBER - AND NOTHING MOVED THIS ROUND. The score of 46 in 961 games is in `data/verification/fix-batch-M6instr-defog.json`, along with the second count of 141. The number our published check actually prints is in `data/game-differential.json` and still reads 77 in 961. Neither was re-measured in this round. If you see the score quoted anywhere from this version, it is being carried over, not re-earned.

WE DID NOT TAKE THE SUMMARY'S WORD FOR ANY OF IT. A first pass had suggested which items were probably dead. Not one item was closed on that suggestion. Each one was closed with its own evidence written into its own line - a real reading from a real tool - dated, with the old wording kept underneath so anyone can argue with the decision later. **22 of the 24** were checked a second time against newer code, because the code moved while we were working. Two of the suggestions turned out to be miscopied, and we wrote that down rather than quietly acting on them.

THE FIX WE HAD BEEN TOLD TO APPLY DOES NOT WORK, AND WE FOUND THAT OUT BEFORE TOUCHING ANYTHING. One item was showing up as broken when its own line clearly said it was finished. The reason is a small piece of text-reading code that gets confused by the `|` character, and the instruction was to put a backslash in front of each one. We tried that on a fake line first: it makes no difference at all, because the reader stops at a backslashed pipe exactly as it stops at a plain one. So we deleted the pipes instead. We also deliberately did not "improve" the shared piece of code that decides whether a line is finished, because we had already shown that you can replace that piece with the word *yes* and **159** of our own checks still pass it. Repairing a tool you cannot trust, to fix a symptom somewhere else, is how a measuring instrument gets a fault nobody can see.

AND IT TURNED OUT NOT TO BE ONE LINE BUT NINETY-EIGHT. 98 lines carry **669** of these stray characters, and **22** of them have their status chopped off as a result. Nine of those read "open" when their own author wrote "closed". **We reported them and left them alone.** Deciding an item is finished because a text bug ate the word is exactly the mistake that started the evening, and doing it ninety-eight times would have been worse. Their authors have to say it again themselves.

FOUR OF OUR COUNTERS WERE COUNTING NOTHING, AND ONE OF THEM WAS PUBLISHED THAT WAY TWICE. We prove that a feature actually ran by counting how often it fires. Four of those counters were adding to a slot that had never been created, which in this language produces "not a number" - and it gets written out as an empty value. One of them was published as empty in `data/million-run.json` and in `data/million-run-150k.json`. The feature fired. The counter said nothing. In two published files. There is an easy patch for this that we did not apply: you can force the slot to start at zero at the point where you add to it. That makes the arithmetic work and hides which object the counter belongs to, and belonging was the whole problem.

SO WE ASKED THE QUESTION OF THE ENTIRE PROJECT RATHER THAN OF FOUR NAMES. A new check reads **507** files and **602** counters and reports exactly **4** problems, with nothing wrongly accused. That last part is the important one: it means we did not need a list of exceptions, and a rule that needs no exceptions is usually the right rule. We ran it against the broken version first and watched it go red on all four, and it says out loud which **6** counters it cannot judge instead of guessing.

ONE MORE, AND IT IS THE EMBARRASSING ONE. We have a guard whose entire job is to notice when a measurement was taken against the wrong version of the engine. The part of it that fires on exactly that condition was never being read. The code that claimed the guard had been proven quoted our own founding rule in its comment and then listed five of its six alarms, missing the one the guard exists for - and it had been written by us, hours earlier the same night. The list is now taken from the guard itself, so a seventh alarm needs no edit.

WHAT THIS ROUND DID NOT DO. It changed nothing about how the game is simulated, it re-scored nothing, it trained nothing, and it does not release any of the numbers we are holding back. The command that regenerates the automatically-written sections of our internal pages has now not been run for three rounds in a row, so those sections are behind. Read the live check, not the page.

5.250.0 - THE SCORE COMES DOWN AGAIN, FROM 53 IN 961 TO 46 IN 961. THE WHOLE EVENING READS 77, THEN 61, THEN 50, THEN 53, THEN 46 - AND WE ARE LEAVING THE STEP THAT WENT UP IN. Same check as always: we play the same 961 real games on our simulator and on the official one, side by side, and count the games where the BOARD ever differs - who is out, at what health, with what status. That count is now 46. A second count, of games where the two versions first say something different at all, went from 154 to 141. Seven games are still thrown out as unmeasurable, exactly as before, and every one of our 829 checked mechanics is still live. **WHY THE SEQUENCE HAS A STEP UP IN THE MIDDLE.** The rise to 53 was a repair working: it stopped four broken games hiding behind a lucky

coincidence, so they became visible and got counted. We could write the evening as "77 down to 46" and it would be true and it would also be edited. A line of numbers that only ever improves is a line somebody has tidied, so ours does not.

WHICH FILE HOLDS WHICH NUMBER. The 46 lives in `data/verification/fix-batch-M6instr-defog.json`. The number our published gate actually prints still lives in `data/game-differential.json` and still reads 77 of 961, because that file was deliberately not rewritten. Both are correct measurements; only one is published. Say which file you are reading from, every time.

WE FIXED THIRTEEN OF FOURTEEN THINGS AND THE FOURTEENTH WAS NEVER ONE OF THEM. The fourteen were CAUSES - individual reasons two games disagreed - and not fourteen games. Thirteen of them were the same thing: a Pokemon that hurts itself in confusion, with the two simulators disagreeing about how much. All thirteen are gone, and that family of disagreement dropped from 22 to 9. The fourteenth only looked like the others: it is the two game logs being at different POINTS in the story, with a confusion line happening to sit at the join. It is a different problem, we did not touch it, and it did not move.

AND THAT IS WHY THE BOARD SCORE ONLY IMPROVED BY SEVEN WHEN THIRTEEN CAUSES CLOSED. Six of those thirteen games have a SECOND thing wrong with them as well, so they stay on the list. Counting reasons and counting games are two different jobs, and subtracting one from the other gives a wrong answer. We say the two numbers separately for exactly that reason.

WE WROTE DOWN WHAT WE EXPECTED BEFORE WE RAN IT, AND WE GOT FOUR OF THE FIVE CALLS RIGHT. THE ONE WE MISSED IS THE HEADLINE. We predicted the board score would land at 39, and it landed at 46. The other four calls were right: the first-difference count, the seven unmeasurable games, the mechanics count, and a call that our Defog repair would change the real-game pool by exactly nothing - which it did, because aiming Defog at your own partner is a play nobody makes. The miss has one cause, and we had already written it down: the "some of these games have a second fault" warning was in the prediction and then not weighted heavily enough. We are recording the miss, because a forecast where only the hits get counted is not a forecast.

ONE HONEST WARNING ABOUT THE BEFORE-AND-AFTER. Half of this repair was to the MEASURING TOOL rather than to the simulator. So the two runs either side of it differ by a changed ruler as well as a changed engine. The count of closed causes is safe from that - a cause is either the same event disagreeing or it is not. The single headline number is not strictly one change against one change, and we are not going to present it as though it were.

THREE PIECES OF CODE THAT LOOKED LIKE COPIES OF EACH OTHER WERE NOT, AND MERGING THEM WOULD HAVE BROKEN A MOVE. We have a rule that a fact about the game gets ONE implementation that everybody calls. Three places choose which side of the field to clear, and they looked like the same choice written three times. They are not. Tidy Up aims at ITSELF, so merging it into the Defog repair would have folded its two lists into one and left the opponent's hazards sitting there. The shared fact is the sweeping; WHICH side each move names is a separate question. This is the rule being correctly NOT applied, and it is written down because the tidy-looking change was the wrong one.

WE DESTROYED FOUR ROWS OF ANOTHER SESSION'S UNSAVED WORK AND WE ARE SAYING SO. A `git checkout --` during this batch wiped four uncommitted notes explaining why certain code is correct. Two were put back word for word from a printout taken before the loss. Two are REBUILDS, labelled as rebuilds inside the file, each quoting the original answer from the old record's own text rather than from anyone's memory, and each asking to be re-read by whoever wrote it. The code they describe has not changed by a single byte and we can prove that by fingerprint, so nothing about the game is now unexplained. But that command on somebody else's unsaved file cannot be undone - git has nothing to give back - and the honest record is a loss, not a repair.

HOW MUCH OF OUR PROBLEM LIST IS ACTUALLY CHECKED BY A TOOL. THE ANSWER IS NOT FLATTERING. We keep 219 open rows describing things believed wrong. 13 of them name a tool that can decide the question and our checker will run it. 43 more say plainly, in writing, that nothing decides them - which is a good thing to admit. Zero now name a tool the checker refuses, down from nine the night before. **That leaves 163 rows that do neither, and 43 of them are rows claiming something is broken.** For those, the gate that keeps our engine quarantined is being held shut by prose. The admitted half got better and the unchecked half got bigger, and reporting only the first would be picking the nicer number.

WHAT IS STILL OPEN. One staged-board test is board-perfect and still reports a failure, on a fault that was already there before this pass and belongs to a different file - reported, not papered over. Two rows we suspect were closed too early are filed with evidence and NOT reopened, because we did not re-run the tool that would decide it. Four closed rows rest on a check nobody has ever actually run. One older defect stays fenced off rather than split up. And the tool that stamps the generated status blocks was not run, so those blocks are one pass behind.

5.249.0 - THE SCORE WENT UP, FROM 50 IN 961 TO 53 IN 961, AND THAT IS THE FIX WORKING RATHER THAN A STEP BACKWARDS. Same check as always: we play the same 961 real games on our simulator and on the official one, side by side, and count the games where the BOARD ever differs - who is out, at what health, with what boosts and status. That count went from 50 to 53. It went UP because of a repair, and here is why those two things are not a contradiction. **WHAT WAS WRONG.** When a Pokemon is confused it can hurt itself instead of attacking. The official game rolls that in two steps - whether it happens, and how hard - and it takes those two numbers from two different places, because the target it is aiming at changes in between. We were taking both from the same place. **WHY TAKING BOTH FROM ONE PLACE MADE US LOOK BETTER THAN WE WERE.** Because our two numbers came from one source, our simulator and the official one sometimes happened to land the same way by pure chance, and a game that genuinely disagreed looked like a game that agreed. **THE REAL SIZE OF THIS PROBLEM IS 14 GAMES. FOUR OF THEM WERE HIDDEN BY THAT COINCIDENCE.** Correcting where the numbers come from removed the coincidence, so four games that were always wrong are now visible and counted. **THE PROOF IS NARROW AND IT IS THE WHOLE ARGUMENT:** only one kind of disagreement changed at all, a field-damage line that went from 18 to 22, and every one of the eight games that moved names confusion as the cause. Nothing else in the entire run changed by even one game. **WE ALSO WROTE THE DIRECTION DOWN BEFORE THE RUN**, as neutral or slightly worse, so this is not an excuse invented afterwards. We are keeping the fix, because our simulator now does what the official one does; undoing it is one setting and that is the owner's decision, not ours.

AND THE BIGGER STORY THIS TIME IS ABOUT OUR MEASURING TOOLS, NOT ABOUT THE GAME. We keep a register of every known problem, and a row can name the tool that checks it. **NINE OF THOSE 124 NAMED TOOLS WERE BEING QUIETLY REFUSED BY OUR OWN CHECKER** - the row said a tool decides this, the checker declined to run it, and nothing anywhere said so. Worse, a refused tool was filed under the same words we use when a tool is broken, so *our ruler would not read this* and *this thing is broken* were the same number. Two of the nine sat on rows that say something is currently broken, which means **just over a fifth of the register's live checking was not real**. We fixed this class of problem once before by listing one wrong way of writing a command; nine more ways walked straight past. So this time the rule is a property rather than a list: there is no shell involved, so only the words before the program name can do anything, that part is refused unless it is understood, and the program has to be inside this repository. Everything after it is harmless by construction. **THE OLD RULE WAS NOT PROTECTING WHAT IT CLAIMED TO PROTECT.** Its own note said it kept slow game-playing runs out of the pass. We measured the old code: the very commands it named were already getting through, one character away. It was guarding spelling, not cost.

THE BEST DECISION THIS PASS WAS TO NOT WRITE SOMETHING. One open row says a thing is broken, and the obvious repair to its marker would have made that row report GREEN while the problem it names sits untouched - the tool it points at prints its 632 refusals into a list that never reaches the pass/fail result. So no marker was written and the row keeps its honest label saying no instrument covers it yet. **A LOUD ALARM TURNED OUT TO BE OUR OWN TOOL LOSING ITS PLACE.** A check that watches which side of the field our simulator acts on went red with four apparently new problems. All four are the identical code we already examined on 2026-08-29, character for character. What moved was the tool's bookmark: a new line was inserted above two of them, and the surrounding function slid 1,660 lines - further than the 1,500-line window the tool searches - for the other two. All four were re-checked against the official game anyway and all four are correct. The count went from 84 to 80 and the check now passes. **BUT A GENUINE PROBLEM WAS FOUND WHILE PROVING THE FOURTH ONE INNOCENT**, in the move Defog, and the old note about it had named the wrong line. That one is written down and still owed.

WHICH FILE HOLDS WHICH NUMBER. The 53 lives in `data/verification/fix-batch-M6-sidesel.json`. The number our gate actually prints still lives in `data/game-differential.json` and still reads 77 of 961, because that file was deliberately not rewritten. Both are correct; only one is published. Say which one you mean, every time. Nothing was retrained, no model figure becomes quotable, and the withheld results stay withheld.

5.248.0 - ELEVEN MORE GAMES STOPPED DISAGREEING WITH THE REAL GAME. THE BOARD SCORE GOES FROM 61 IN 961 TO 50 IN 961 - AND THE PUBLISHED NUMBER IS STILL 77. Same check as the block below: we play the same 961 real games on our simulator and on the official one, side by side, and count the games where the BOARD ever differs - who is out, at what health, with what boosts and status. Across tonight that count has gone 77, then 61, then 50. The separate commentary count - the words a battle prints, which we track apart from the board and which does not decide anything - went 161 to 150. The count of games we throw away as unusable did not move. The laboratory count of staged mechanics stayed level throughout. Both runs used the same frozen simulator rules, the same 961 games and the same frozen team pool, so the three fixes in this version are the only thing that changed. **THE THREE FIXES.** (1) Sucker Punch is a move that only works if the target is about to attack; when an ally had pulled the attack onto itself, we were asking the wrong Pokemon that question, so the move went through when the real game refuses it. (2) The repeated-Protect die - the one that makes Protect less and less likely to work when you spam it - was being rolled as two separate coins instead of the one shared coin, in the case where the Pokemon had been forced to repeat the move. (3) An ability that punishes the attacker on contact was announcing itself on the wrong Pokemon. **WE WROTE DOWN FOUR PREDICTIONS BEFORE THE RUN AND ALL FOUR CAME IN.** That is worth stating plainly next to the block below, where we called three and hit two, because the value of writing predictions down is only real if the misses are counted as well. **AND THE MOST USEFUL THING THIS PASS FOUND WAS A TEST THAT PASSED FOR THE WRONG REASON.** The first version of the third test had the Pokemon clicking Protect, so the contact attack never happened on EITHER simulator - two boards agreeing about a mechanic that never fired. Nothing about the boards could have shown that. It was caught by a counter that says how many times the mechanic actually ran. **A COMMAND IN OUR OWN INSTRUCTIONS ALSO DID NOTHING AND REPORTED SUCCESS**: asking for an output file without asking to write one produces no file and exits cleanly. That is the ninth time we have found that shape of mistake, and this time we wrote it ourselves. **AND WE LOOKED AT ABOUT SIXTY CHECKS THAT NOTHING EVER RUNS AND SWITCHED NONE OF THEM ON, WHICH IS THE RIGHT ANSWER.** Almost all of them load the simulator, and the simulator was being edited while we looked. Certifying a check against a moving target proves nothing. **TWO REAL PROBLEMS FOUND AND NOT BURIED.** A check on how our simulator picks which side of the field to act on is failing and nobody had run it; all of the new cases arrived in earlier work. And nine of our tracking notes name a tool to verify them that the tool itself refuses to run, so those rows read as never

started - a problem we fixed once before and fixed too narrowly. **STILL OPEN, SAID PLAINLY:** the gate is still red at 50 games; one clause of an earlier fix is not closed; a form-change counter that did not move is about timing rather than changing back; the largest unexamined disagreement is still the accuracy case involving Parting Shot; and a fourth planned fix is deliberately untouched until it can be split into pieces small enough to attribute. Freezing a new copy of the simulator also left several of our pinned reference measurements naming a version the code has moved past - they have to be re-measured before they can speak, which is the guard doing its job rather than something breaking. Nothing was retrained, no model figure becomes quotable, and the withheld results stay withheld.

WHICH FILE HOLDS WHICH NUMBER, FOR THIS VERSION. `data/verification/fix-batch-M5M7M8.json` holds the new board score of 50 out of 961 games. `data/game-differential.json` is the one our gate reads, it was not rewritten, and it still holds 77. Both were measured correctly. Only the second one is published. Say which one you mean, every time.

5.247.0 - SIXTEEN GAMES STOPPED DISAGREEING WITH THE REAL GAME. THE BOARD SCORE GOES FROM 77 IN 961 TO 61 IN 961 - AND THE NUMBER THAT IS PUBLISHED IS STILL THE OLD ONE. To check our simulator we play the same 961 real games on it and on the official one, side by side, and count the games where the BOARD ever differs - who is out, at what health, with what boosts and status. That count was 77. After this version's three fixes it is 61. **BOTH NUMBERS ARE CORRECTLY MEASURED AND ONLY ONE OF THEM IS PUBLISHED.** The new one was written to a separate verification file; the file our gate actually reads was deliberately not rewritten, so the gate still says 77 until somebody republishes it. Anyone quoting a whole-game figure has to say which of the two they mean - having two right answers in the room with only one of them published has already cost us three arguments. The separate commentary count - the words a battle prints, which we track apart from the board and which does not gate - went from 168 to 161. Both runs used the same frozen simulator rules, the same 961 games and the same frozen team pool, so the three fixes are the only thing that changed. **THE THREE FIXES.** (1) Moves that strike several times and roll accuracy for EACH strike were landing the wrong number of strikes. There are exactly two such moves in this format, Triple Axel and Population Bomb, and we read that from the format rather than remembering it; the real game rolls per strike and stops at the first miss. (2) A Pokemon whose changed form is only temporary was not changing back when it left the field. (3) A Pokemon locked into one move never had that lock cleared. **HOW WE GOT THERE IS WORTH MORE THAN THE NUMBER.** Our own diagnosis notes say WHERE a problem is and WHY it happens, and the why was wrong twice out of three. The multi-strike problem was not about which random number we drew - we were drawing the same one as the real game, all eleven of eleven times - it was about the accuracy value we drew against, and we proved that with arithmetic BEFORE changing a line. And two of the three Pokemon we blamed for the form problem were already correct. That is the pattern for thirty-two batches running: **a diagnosis is reliable about where and unreliable about why. WE WROTE DOWN WHAT WE EXPECTED BEFORE THE RUN AND SCORED TWO OUT OF THREE.** We called the board score at 60–70 and it came in at 61. We called the commentary at about 162 and it came in at 161. We called a third counter dropping to 0–2 and it did not move at all. **The miss is written down on purpose**, because a prediction whose misses are not counted is not a prediction. **AND ONE MISTAKE WAS CAUGHT BY THE OTHER SCOREBOARD.** Our first attempt at the third fix quietly broke a staged mechanic - our laboratory count went 829 to 828 - while the 961 real games scored exactly the same either way. We keep two scoreboards for that reason, and only one of them could see it. **STILL OPEN, AND NAMED RATHER THAN GLOSSED:** a form-related counter did not move at all, because what is left there is the TIMING of the form flip rather than the changing back, and nothing tests it yet; the largest remaining unexamined disagreement is an accuracy case involving Parting Shot; and a fourth planned fix is deliberately untouched until it can be split into pieces small enough to attribute. Nothing was retrained, no model figure becomes quotable, and the withheld results stay withheld.

WHICH FILE HOLDS WHICH NUMBER. `data/verification/fix-batch-M1M3M4.json` holds the new board score, 61 of 961. `data/game-differential.json` is the one the gate reads, it was not rewritten, and it still holds 77 of 961. Both were measured correctly. Only the second one is published. Say which one you mean.

5.246.0 - THE SIMULATOR BUG WE ANNOUNCED HOURS AGO IS NOT REAL. THE TEST THAT FOUND IT WAS CHECKING ITSELF. The block below tells you that *"our simulator gives that doubling to every Pokemon that loses an item, whether or not it has the ability"* - that anything which lost its held item was wrongly moving at double Speed. **We are withdrawing that.** The old wording stays exactly where it was published, because we do not quietly rewrite something we told you yesterday; this note replaces it. **WHAT THE CODE ACTUALLY DOES.** The line we quoted is only the door - it asks whether this Pokemon started with an item and is now empty-handed. The doubling happens on the NEXT line, and that line first asks whether the Pokemon's ability is the one tagged for it. Exactly one ability in the whole format carries that tag. A check we had already run says it plainly: a Pokemon with no ability at all comes out of losing its item at the same Speed, `187` and then `187`, while the one with the ability goes from `187` to `374`. **WHY WE GOT IT WRONG IS THE PART WORTH KEEPING.** The test never looked at our simulator's Speed at all. It worked out for itself whether each Pokemon had lost an item, and compared THAT against the real game's record of who had the speed boost. So when it reported a match, all it was really saying was *both of these lost an item*. It could not have told us anything about Speed, in either direction. **A tool can be broken before the thing it is testing is** - and that was the third time in one night here. **SOMETHING IS STILL WRONG, AND IT IS SMALLER.** We work the doubling out fresh, from whatever ability the Pokemon has right now. The real game hands the boost out at the moment the item goes and then remembers it. So a Pokemon that is GIVEN the ability after its hands are already empty - Skill Swap is how that happens in a real game - gets double Speed with us and not in the real game. That is written down as a numbered job, with the note that **nothing currently tests it**. **WE ALSO WROTE DOWN SIX PROBLEMS THAT UNTIL NOW EXISTED ONLY IN PROSE**, in the place where jobs are tracked rather than in a paragraph somebody has to remember. Only one of the six has a measurement behind it; the other five say plainly that the tool which would decide them does not exist yet. Nothing was retrained, no result changed, and no number on this deck moved.

5.245.0 - OUR MAIN CORRECTNESS CHECK WAS COUNTING THE COMMENTARY AND NOT THE BOARD. THE HONEST NUMBER IS 77 GAMES IN 961, AND THE 167 WE HAVE BEEN QUOTING IS A DIFFERENT THING. To check our simulator we play the same 961 games on it and on the official one, side by side, and look for the moment they disagree. **There are two completely different things you can count there, and the check was counting the wrong one.** The **commentary** is the running text the game prints - who moved, what fainted, which message appeared. The **board** is the actual position: who is out, how much health is left, what is boosted, what is asleep. Will's ruling in August was that the commentary is allowed to differ and the board is not, and this version is the first time the check actually works that way. **The number that now decides whether we are finished is 77 games out of 961 (8.0%)** - the games where the two simulators end up holding a different position. The commentary number, **167 of 961**, is still measured, still printed and still work we owe; it simply no longer holds the gate shut, and it keeps its own row, its own count and its own failing exit code so that it cannot quietly become a backlog. **We will never print either number again without saying which of the two it is. THIS IS NOT US MOVING THE GOALPOSTS, AND THERE IS A MEASUREMENT THAT SAYS SO.** Of the 167 commentary disagreements, **102 never change the position at all** - they are wording. Going the other way, **11 of the 77 broken boards had no commentary disagreement whatsoever**: the position quietly went wrong and nothing in the text pointed at it. Under the old single count those 11 games were not counted anywhere, which means **a fix could have improved our headline score without improving the simulator at all.** They are now printed as their own kind. The gate overall reads CLOSED, with one of its eight gating checks failing - and that one failure is real engine work, not a paperwork problem. **THE**

COMPARISON WAS ALSO AGREEING BY NOT LOOKING. Three states a Pokemon can be in - unable to use a sound move, recharging after a huge attack, and powered up by having absorbed a Fire attack - were being written onto the board with nothing reading them, so a game whose only disagreement was one of those was scored as a match. All three are now compared. We proved each one first by planting a deliberate difference and watching the old comparison miss it, and we ran the same game with no difference planted to be sure we had not simply made the check noisy. **Then we re-ran the 961 games and nothing moved: 77 before, 77 after.** We said in advance that the number could only rise or stay the same, and it stayed the same. That does not prove those three are clean - it says nothing was hiding in them in these particular games. **AND ONE CORRECTION WE OWE FROM EARLIER THE SAME DAY.** We said this widening would generalise to the rest. Half of that is true: the comparison side is two lines per case. **The test fixtures are not** - each one has to be built by hand against the real game's own refusals. One case, Unburden, looked ready on every automatic signal and turned out to be something else entirely, and finding that out found a **real bug in our simulator**. Unburden is an ability that doubles your Speed when you lose your held item; **our simulator gives that doubling to every Pokemon that loses an item, whether or not it has the ability.** Knocked off, eaten berry, spent Focus Sash - all of them wrongly twice as fast. That is filed as an engine defect, and the nineteen remaining cases are real work rather than a loop. **HOUSEKEEPING THAT MATTERED MORE THAN IT SOUNDS.** Five of our pinned reference measurements were regenerated and now carry proof of exactly which version of the engine produced them; **not one measured result changed.** Three of the five could not be regenerated at all until we repaired a guard added hours earlier that was killing two of our own scripts before they played a single game - and killing them with the exit code that means *skipped*, so the test runner would have reported them politely skipped rather than failed. We also archived Smogon's August 2026 usage statistics as a comparison set that feeds nothing we decide with: 310 Pokemon, 1,269,250 games, zero illegal entries. **Nothing was retrained, no model changed, and nothing currently withheld becomes quotable.**

5.244.0 - EVERY PROBLEM WE FOUND THIS TIME, THE PROJECT HAD ALREADY WRITTEN DOWN SOMEWHERE THAT NOBODY WAS READING. Nothing in the simulator changed this version. Nothing was retrained. What changed is that we built things that read what our other tools were already saying. **The three that started it.** Our status report was printing the line that explains last version's whole mess - it sat at line 103 of a 253-line printout. Our replay collector had a comment in it explaining the archive problem and telling the reader, in capitals, to run something before ever re-parsing. Our test runner was failing its own check that every test is accounted for. None of this was hidden and none of it was wrong. It was simply never in front of anyone. **So the honest question stopped being "which bug".** It became: does a fix make the whole class of failure impossible, or does it shut one door and leave the next one open? Every repair below is labelled one or the other, and the test is - would this catch the same mistake made a slightly different way, somewhere else? **The new tool: what does each of our instruments fail to check about itself.** It runs in about three seconds and refuses to pass if it finds anything. First run: **60 checks that nothing ever runs**, **3 of the 8 clauses in our main gate cannot tell when their own input has gone stale**, **12 published figures out of date**, **783 of 1,048 counters that nothing reads**, and **614 rows in the store with no raw log kept**. We deliberately broke each of its five sections first and confirmed it went red, because a tool that quietly measures nothing reports a spotless project. **We tested our tests by sabotaging them on purpose.** For the simulator we have a referee - the official game - so we can always ask who is right. **For our own checks there is no referee**, which is exactly how a gate reading a perfect eight-out-of-eight survived being wrong. So we made **83 small deliberate breakages** in the gates and watched: **57 were caught, 26 went unnoticed**. Those 26 are clauses that could fail without anybody finding out. **The big repair: we had it backwards about which file matters.** The raw replay log is the thing we can never get back. The tidy store built from it can be rebuilt any time. We were keeping the rebuildable one and, in one case, **deleting the irreplaceable one** - a game our current parser could not read had its raw log thrown away, which is exactly the game a better parser

tomorrow would have wanted. We also wrote the tidy row before the log, so a crash left a row with no log instead of the other way round. Both are fixed, and **we recovered 7,275 raw logs across the two stores - 6,661 and 614 - with none missing and none undated.** Re-parsing is unblocked. **A reversal: we said the archive was about 65 days from trouble and it is not.** Measured, it compresses to **13.98%**, and a single archive file is **78 MB today across both stores - already 78% of the 100 MB limit** we would hit. So the archive is now written as small dated files that are never rewritten. While testing that we caught a nasty one: a "-2" suffix used to avoid a name clash sorts before the plain name, which would have replayed the archive out of order. **A second reversal: last version's fix to the damage-table alarm was not enough, and we said it was.** It repaired two named fields. But the alarm still worked off a hand-written list of what to check, so anything added tomorrow would be unwatched again - one door closed, the class of problem untouched. The list is gone; the alarm now works out for itself what to check. Doing that immediately found something the list could never have: **a field that is missing looks exactly like a field that is empty**, so deleting a Pokemon's entire move list moved nothing at all. That was live on **4 rows for moves and 10 for items**. The alarm's reading did not change, so nothing new is owed because of it. **One of our checks had five silent limits, not one.** It looked at 8 things where there are far more, stopped 3 levels deep where the deepest is 10, never went inside lists, and quietly passed if the file it was checking failed to load. Between them, a table of **230 entries was never once inspected** - inside the very check written to catch typed-in-by-hand mistakes. The reason given for the shortcuts was speed, and it was false: checking everything takes **733ms against 596ms**. **We broke exactly one thing, and we proved that rather than claiming it.** The full run is **143 passing and 28 failing**. We re-ran every one of those 28 against a separate copy of the project from before this session's work: five are tests that assert something that has since changed, three could not start at all for lack of memory, twenty were already red, and **one is ours**. Two failures that looked like the simulator breaking were cleared by that control. **Even the count of failures was wrong twice before it settled** - 23, then 30, and the truth is 28. The first was us counting lines of the runner's output while it was still being written. **And one command ran for twenty minutes having done nothing at all.** It looked busy. The giveaway was that it had used **2 seconds** of processor time. **We also counted our comments for the first time: 495 files, 17,519 comments.** Not one was found to contradict its own code, which is a better result than expected. But **37 refer to things that no longer exist**, and the standout is a section headed "pruning, and why it is safe" whose argument rests on there being **nine saved copies taking 23 MB**. There are **523 taking 2.5 GB**. **What we did not do.** No model was retrained, the weights file was not touched, and MAG stays paused. Nothing that was withheld becomes quotable. The generated status blocks in our internal ledgers are one pass behind on purpose. **5.243.0 - OUR MAIN CORRECTNESS CHECK WAS MARKING THE SIMULATOR AS FINISHED USING GAMES THAT NEVER FINISHED.** We have one big gate that decides whether our simulator plays Pokemon the way the real game does. It read a perfect score - all eight of its checks passing. It now reads CLOSED, with one of the eight failing, and that is the honest reading rather than a step backwards. Nothing was adjusted to make it fail. **What went wrong.** One of those eight checks plays a batch of games twice, once in the real Showdown engine and once in ours, and asks whether the two ever end up in a different position on the board. The file it was reading came from a driver whose job is to touch as many rare mechanics as possible, not to win. Out of 961 games it played, only 17 reached a winner. The other 944 hit our twelve-turn stopwatch with everybody still standing. So of course nothing ever came apart: **you cannot disagree about how a game ended if the games do not end. The proof that it was the driver and nothing else.** We ran it again with everything held identical - the same frozen copy of the simulator, the same twelve-turn cap, the same frozen set of real ladder teams, the same list of mechanics to watch, the same 961 games out of the same budget - and changed only the driver, to one that copies what real human players actually clicked. That driver finishes 474 of the 961 games, close to half of them, and **77 of them end up in genuinely different positions. The honest number, and there are two of them.** 77 games out of 961 - 8.0% - is where the two engines really part company on the board. A second and larger count of 168 is where

their running commentary first differs, which is not the same thing and often does not matter to anybody. We will say which of the two we mean, every time. **This reverses something we published.** Anywhere we have said this check was clean, or that the simulator was close to being signed off, we were quoting the games-that-never-end run. Those older write-ups are left exactly as they were and marked superseded here, because quietly rewriting a past conclusion is the one thing we do not do. The simulator is further from finished than we said it was. **What we fixed and what we deliberately did not.** The check now refuses a file from the wrong driver outright instead of quietly passing on it, and the tool that produces the file can no longer publish a coverage run into the published slot. We did NOT change what the failing check counts, even though we know it is counting the larger commentary number rather than the board number - changing what is counted in the same breath as turning the gate red would look exactly like moving the goalposts, and that is the owner's call. Nothing that depends on the simulator was retrained or re-run; our move-choosing model stays paused, exactly as it was. **5.242.0 - THE ALARM THAT WATCHES OUR DAMAGE TABLE WAS NOT WATCHING TWO OF THE THINGS IT SAID IT WAS.** Our move-choosing model is trained against a table of every Pokemon - stats, typing, weight, moves. If that table changes, the trained model may be out of date, so there is an alarm whose whole job is to notice the table moving. It boils the table down to a short fingerprint and compares it against the fingerprint saved when the model was trained. **It was asking for a column that does not exist.** It asked each row for a typing field spelled `ty`, and this table has never spelled it that way - typing lives in a field called `t`. All 322 rows answered "nothing", every time, so a Pokemon changing type would never have moved the fingerprint. Weight was worse: it was not looked at at all. Weight is not decoration here - six things in this format read weight or change it: Low Kick, Grass Knot, Heavy Slam, Heat Crash, Heavy Metal and Light Metal. A weight edit could have changed real damage and left the alarm silent. **How we proved it, because "we read the code and it looks wrong" is not proof.** We changed one Pokemon's typing in memory and asked whether the fingerprint moved. It did not. We changed a weight. It did not. Then we changed two fields we already knew were being watched, and the fingerprint moved both times - that is the control, and without it the whole test would only have shown that our own probe was broken. After the fix, all four move. One more trap was worth naming: a field that is MISSING looks exactly like a field that is there and empty, so we also invented the missing field on a single row and watched the fingerprint move before the fix. That proves the alarm was wired up correctly and the data had simply never filled it in. Only a test that changes something can tell those two apart. **What we did NOT do, on purpose.** Fixing the alarm changes the fingerprint, so the saved one no longer matches - but it already did not match, because the table itself was rebuilt from 318 to 322 Pokemon days ago. Re-saving it now would fix both at once, and nobody could ever say afterwards which of the two was the real reason, so we left it alone and wrote the distinction down. And we did not re-train the model: the owner's standing decision is that it stays paused until the simulator underneath it is correct, and re-training on a simulator we know is still wrong would only mean doing it again. That is a decision, not something we forgot. The warning light is still on, and it is still telling the truth. Full account: `docs/_reports/2026-09-03-table-digest-blind-fields.md`. **5.241.0 - AN ATTACK THAT LANDS TWO TURNS LATER COULD NEVER LAND A CRITICAL HIT, AND A HIT THAT SMASHED A SUBSTITUTE CHARGED US FOR THE WHOLE SWING.** Some moves are set up now and land two turns later. In the real game that delayed hit is an ordinary hit in every respect - it can be a critical hit like anything else. Ours never rolled for one, ever. **The way we caught it matters more than the bug.** We did not go looking for a rare event; we handed both simulators a loaded die that MUST give a critical hit, then a die that CANNOT give one. The official game answered 72 damage with the critical-hit message and then 48 without it. Ours answered 69 both times. A number that does not move when you change the only thing it depends on is not a small error - it means the wire was never connected, and no amount of playing games would have found it. It is connected now, and it is applied at the same point in the damage sum the real game uses rather than multiplied on at the end, which would have been the right number in the wrong place. **The second fix is about Substitute** - the doll a Pokemon puts in front of itself to soak a hit. If your attack is far stronger than the doll, the real game only counts the part the

doll actually absorbed. That matters for two families of move: one that hurts you back for a share of what you dealt, and one that heals you for a share of what you dealt. We were counting the whole swing. A recoil move that should have cost 12 health cost 25, and a draining move that should have healed 21 healed 62 - so a Pokemon could break a doll and come out of it healthier than the real game allows. There is now one shared piece of code that answers "how much did that actually deal", instead of two that could drift apart. **One thing we are NOT claiming, deliberately:** the machine was in use, so we did not re-run the comparison against real ladder games. We are not saying it improved and we are not saying it stayed still - the measurement was not taken, and an untaken measurement gets left out rather than written down with an excuse next to it. Our checklist of things the simulator can prove it does correctly went from 827 to 829. **We also settled a question that had been sitting on our own dashboard as a red warning.** The table of Pokemon data underneath the move-choosing model was rebuilt, and the question was whether that forces us to re-train the model. It does not, and we measured why rather than arguing it: every row that changed is a mega evolution or a mid-battle form, none of the stats, types, items or abilities moved on any row, and the one substantive change - 76 megas getting real movesets - never reaches the model at all, because the model overwrites those moves with what the team sheet actually declared. Out of 16,830 games, the number that could be affected in any way is 12, or 0.07%. So the warning gets a re-stamp, not a re-training. **And we found a hole in the alarm itself, and did not fix it yet.** The check that is supposed to notice when that table changes has been looking at a field that does not exist on any of the 322 rows, and it never looked at weight at all - so a Pokemon's type or weight could change and the alarm would stay silent. It is written down and it is deliberately left for the same pass as the re-stamp, because fixing it changes the very number the re-stamp records. **5.240.0 - ELECTRIC TERRAIN IS SUPPOSED TO KEEP YOUR POKEMON AWAKE, AND OURS LET THEM BE PUT TO SLEEP ANYWAY.** The terrain that crackles across the ground stops anything standing on it from falling asleep - that is one of the two reasons people set it. Our simulator knew the half that raises the power of Electric moves and knew nothing at all about the half that refuses sleep, so a Sleep Powder or a Yawn landed on it exactly as it would on an empty field. Both halves are now wired, and they had to be two separate pieces of work rather than one: Yawn does not put you to sleep on the spot, it hangs a countdown over you, and a fix that only stopped the sleep two turns later would have left that countdown - and the message announcing it - sitting on the board for nothing. A body lifted off the ground, by Levitate or by being a Flying type, is still fair game, and the terrain refuses SLEEP and not every status, so a burn still lands. Rest, which puts its own user to sleep to heal, now fails outright on the terrain and heals nothing - which is what the real game does and 346 Pokemon in this format can click. **We also found and did not fix two of our own test setups that use a Pokemon and a move this regulation does not allow** - reported rather than quietly corrected, because they belong to another piece of work. Our checklist of things the simulator can prove it does correctly went from 825 to 827. On 961 real ladder games nothing moved at all, and we said so in writing before the run: this is a rare situation and the games we sample never reach it - we measured that directly rather than guessing, by counting how often the new rule fired in those games. It fired zero times. **5.239.0 - ONE ABILITY WAS SCATTERING ITS CALTROPS ON OUR OWN SIDE OF THE FIELD, AND ANOTHER NEVER KICKED UP ITS SANDSTORM.** Glimmora's Toxic Debris throws poison spikes onto the ATTACKER's half of the field when it is hit by a physical move. Our simulator worked that out from who threw the punch - which gives the right answer when the punch comes from the opponent, and the WRONG answer when your own partner hits you with something like Earthquake, because then it dropped the spikes on your own side and poisoned your own switch-ins. Sandaconda's Sand Spit is simpler and was worse: it is meant to kick up a sandstorm whenever it is hit, over the top of whatever weather is already out. We only let it work from a clear sky - so against a sun or rain team, which is exactly when you bring it, it did nothing at all. We had been told these were probably the same underlying problem. **They are not.** They are two separate mistakes in two lines that happen to sit next to each other, and we proved that by building two separate off-switches and showing that each one breaks only its own case and leaves the other alone - and that a third ability in the same family, Rough Skin, does not

move at all either way. Against the official game, on 961 real ladder games: the number of games where the two simulators end up holding a different board dropped from 82 to 80, and the disagreement list lost exactly the two entries that name these two abilities. Our internal checklist of things the simulator can prove it does correctly went from 819 to 821. **5.237.0 - TWO ACCURACY EFFECTS AT ONCE WERE BEING MULTIPLIED TOGETHER, AND THE REAL GAME ADDS THEM UP FIRST.** If your attack's accuracy has been lowered a step and the target has raised its evasiveness a step, the real game combines those into a single net step and looks up one number: **60%**. Our simulator applied each one separately and got **56.25%**. At the extremes it was worse - **11%** where the real game says **33%** - and in one common shape it went the other way, giving **80%** where the truth is **75%**, so this was never simply "we were too pessimistic". It decides whether an attack hits, so it matters. We also found something we were not looking for: the real game throws away the fraction at the end, and we kept it. That means the two versions disagree even when only ONE of the two effects is in play - an 80%-accurate move into a maximally evasive target is **26%** in the real game and 26.67% in ours. That mattered immediately, because one of our own existing tests had written down 26.67% as the correct answer, so the test was quietly protecting the bug. It is corrected, and not from arithmetic: **we instrumented the official simulator and read the number it actually rolls against.** The new test was shown failing first, on five deliberately varied setups, alongside four boring cases that must NOT change and did not. Nothing else moved - the 961-game comparison against the official simulator is identical in every single figure, which was predicted in writing beforehand, because raising evasiveness is genuinely rare in the games we sampled: 40 of 13,214.

5.236.0 - KING'S ROCK REALLY IS A "super flinch machine" ON A MULTI-HIT MOVE, AND OUR SIMULATOR WAS NOT PLAYING IT THAT WAY. King's Rock gives an attack a 10% chance to make the target flinch - lose its turn. The question was what happens when the attack hits several times in one go, like Population Bomb's ten strikes. **The real game rolls that 10% separately for every strike that lands.** Ten strikes is not a 10% chance of a flinch, it is about 65%. Our simulator rolled once for the whole attack, so it treated the item as a flat 10% no matter what. That is not a small rounding difference: it is the difference between a niche item and one of the more oppressive things in the format, and **82 of the 211 King's Rock sets in our stored games are built on exactly this combination.** We now roll once per strike that lands. Two details we checked rather than assumed: the game stops rolling the moment the target faints, so a volley that kills on its third of ten strikes rolls three times and not ten - we match that; and the item deliberately adds nothing to an attack that already flinches on its own, which we also match. **We did not just read the rules, we instrumented the official simulator and counted its dice**, then made ours produce the same count. The test that proves it was shown failing first, and it comes with four deliberately boring cases that must NOT change, so a fix that over-corrected would have been caught. Nothing else moved: the 961-game comparison against the official simulator is identical in every figure, which was predicted in writing before the run, because this build is common on paper and rare in the games we sampled.

5.235.0 - A NEW REGULATION LANDS IN ABOUT TEN DAYS, AND THE COLLECTOR NOW STARTS PULLING IT ON ITS OWN. Pokemon Showdown only keeps the last ~1,250 replays per format on its public site, and on our busiest ladder that is about eighteen hours' worth. Anything older is gone for good. So the opening days of a new regulation - the part everyone most wants to see, before anybody has worked out what is good - are the part you cannot go back for. Waiting until somebody notices the format exists and edits a config file costs exactly that. **We have not guessed what the new format will be called, because a guess written into a file looks exactly like a fact.** Instead the collector asks Showdown's own server, every six hours, which formats it currently runs, and recognises a Champions regulation by its SHAPE rather than its name. The day the new one appears, collection starts on the next run with nobody touching anything. **Today it collects nothing, and it says so in words.** That matters more than it sounds: a tool that does nothing because there is nothing to do, and a tool that was never wired up, look identical on every day except the one that counts - which is this project's oldest and most expensive

failure. So the run prints what it found, what it decided, and a zero where the zero is right. A different zero - finding NO Champions formats at all - is flagged as an error, because there has never been a moment when none existed, so that zero means our own detector broke. **Each regulation gets its own file, named after the format, so two regulations can never be mixed together. And a real bug fell out of building it:** every game we store was being tagged with the CURRENT regulation's name no matter which regulation it came from - a hardcoded label. The one alarm we had for "the format has changed" reads that label, so it was comparing a constant against itself and could never have gone off. We now read the regulation off the battle log. Games from the current regulation are tagged exactly as before, checked against 800 real stored games, so nothing already collected changes meaning.

5.234.0 - OUR OWN DIVERGENCE REPORT WAS MARKING REAL PROBLEMS "CANNOT HAPPEN IN THIS FORMAT", AND TWO OF THEM WERE PROBLEMS THAT CHANGE WHO WINS. When our engine disagrees with the official simulator we write down what the disagreement was about, and we tag each one with whether the Pokemon, move, item or ability involved is even legal in this format - so nobody spends an afternoon on something no real game can reach. The tag was being worked out by looking the name up in the game's data and taking the first answer, and **a name can mean more than one thing.** Three examples, and all three were live: the *condition* Heal Block, which a perfectly legal move called Psychic Noise applies, shares its name with a *move* that is not in this format; the Metronome *item*, which is legal, shares its name with a *move* that is not; and Floette-Mega, which people actually bring, is written on the battle log simply as "Floette" - and the plain Floette that name resolves to is not legal. **All three were being labelled impossible, and anyone triaging by that label closed them without looking.** Two of the three - the Floette ones - are disagreements where the boards genuinely end up different. **This is the worst kind of bug we get here, because nothing goes red.** A tool that invents a problem makes noise; a tool that hides one makes silence, and silence looks exactly like success. **The fix asks the game rather than keeping a list.** We now work out, from the format itself, every condition that something legal can actually cause - 96 of them - and check which of those share a name with a move that is banned. Three do. We do the same for items and for Pokemon formes. Nothing in the fix names a Pokemon or a move, so **the next collision, whatever it is called, is caught with no change to the code. We proved it by turning it off.** The new test has a switch that puts the old behaviour back; with the switch on it fails six ways, with it off it passes. And it re-reads our real measurement file and reports the label change directly: three rows relabelled, none newly hidden. **Nothing about the game itself changed.** This is a change to a measuring instrument, not to the engine - the frozen engine snapshot is the same before and after, and every headline number is untouched.

5.233.0 - TEN POKEMON FORMES HAD NO WEIGHT AT ALL IN OUR DATA, AND FOUR MOVES IN THIS FORMAT ARE PRICED ENTIRELY BY WEIGHT. Low Kick and Grass Knot hit harder the heavier the target is; Heavy Slam and Heat Crash hit harder the heavier the attacker is compared to the target. Ten formes - five megas, a battle-only blade forme, the three Gourgeist sizes and one battle-only water forme - had no weight recorded, and that fails two different ways. If a Pokemon TRANSFORMS into one of them mid-battle it keeps the weight of what it used to be. If you simply BRING one, it has no weight at all, and the move falls through to a base power of zero: **one damage where the real game deals fifty-five.** The generator that writes our data file was fixed last night and the file itself was deliberately left alone, because regenerating it is a decision - it is frozen into every measurement we take. **This is that decision, and it turned out to be two changes rather than the three we were warned about.** The third was a new row for a Pokemon we already had a row for, under a different spelling. Our own artifact audit caught it by name the moment it landed and said exactly what it costs: two rows for one body will eventually disagree, and the emptier one wins. So the generator now drops a duplicate by asking the game which species a row actually is, rather than by us naming anything - and we printed what that rule matches before switching it on: exactly one, the one in question. **The other change was a reordering of the rows, and reordering is**

only safe if nothing reads them in order. We checked rather than assumed, found one place that genuinely does read them in order and would break on a tie, showed that this particular reordering never reaches it, and then proved the whole thing by building a control file with the old numbers in the new order and running all 815 of our mechanic checks against it: **not one verdict changed. We wrote down what we expected before measuring anything, and got six of eight exactly right and eight of eight inside their stated range.** Our lab count went 814 to 815 checks passing. Against 961 recorded human games the disagreement count went 173 to 172 and the number where the actual board differed went 83 to 82 - one game, and we can name it: a Heat Crash at a Falinks that had just mega evolved. **The one thing we got wrong is worth saying plainly.** We predicted which Pokemon could be affected and listed six. Falinks was on our list of Pokemon that could NOT be affected - it gains weight when it mega evolves but not enough to cross a threshold - and we used it as the control in our own test for exactly that reason. It was right about the first pair of moves and wrong about the second pair, which compare two weights against each other rather than reading one off a table. The measurement found the case our arithmetic had missed, which is what a measurement is for.

5.232.0 - TWO MOVES STEAL THE OPPONENT'S BERRY, AND WE WERE THROWING IT AWAY INSTEAD OF EATING IT. Bug Bite and Pluck take a berry off whoever you hit - and in the real game the ATTACKER then eats it. Our simulator took the berry and stopped there, so a Scizor that bit a Sitrus Berry off a Farigiraf healed nothing, when the real game heals it a quarter of its health. It also announced the theft with the wrong wording, which is how a comparison against the official simulator spots it in the first place. **Both halves are fixed, and we checked there were only two such moves rather than trusting the two we were handed** - of the nine legal moves that can take an item, exactly these two make the thief eat it. **The five things that must NOT change are the point of the test**: Knock Off takes the same berry and the attacker must still get nothing; a Sticky Hold holder keeps its berry entirely; an empty hand gives nothing; a Life Orb is not a berry and is not even taken; and the thief must not be able to Recycle a berry that was never its own. All five are identical before the fix, after it, and with the fix switched off. **We wrote down what we expected before running anything, and for the first time in six batches we predicted the real-game measurement would move - and it did**: three of 961 recorded games disagreed with the official simulator on this exact line, and after the fix two of them run perfectly and the third runs further before disagreeing about something else. One game's board is now correct that was not: a Scizor that should have finished on 125 health was on 89. Seven predictions, five exactly right, all seven inside their stated range. **One related thing is deliberately NOT fixed and we say why**: Ripen, an ability that doubles berry effects, should halve an incoming hit twice and only halves once. That needs a rebuild of our derived rule table, which is blocked because the game store is being appended to every few hours and the rebuild would sweep in unrelated changes. It is filed with a measurement, not with a shrug.

5.231.0 - AN ABILITY THAT HEALS YOU FOR EATING A BERRY WAS NOT FIRING ON THREE OF THE SEVEN WAYS A BERRY GETS EATEN. Cheek Pouch heals a third of your health whenever you eat a berry. In our simulator it only noticed berries eaten one way - the ordinary "I am low, I will eat my Sitrus" way. It missed a berry eaten to soften an incoming attack, a berry eaten to cure confusion, and a berry thrown at you by the opponent. In every one of those the Pokemon ate the berry and got nothing. Fixed, and it changes real health totals: on a test board the same Pokemon now ends on 725 health instead of 328. **The bigger point is that we did not stop at the three we were told about.** We asked the game itself where else it announces "a berry was eaten" and got TEN places, not three. Four already worked. Three are the fix. **The other three we deliberately did NOT build, and each was refused with a measurement rather than a judgement call** - one of them can only ever reach its own ability, which the game then immediately clears, so building it would be a counter attached to nothing; the other two need a Pokemon that both has one of these abilities and knows one of two specific moves, and we checked every legal Pokemon in the format and none of them do. **We also fixed four other abilities on the way, and that**

is not luck - the real game does all its berry bookkeeping in one straight line, so a road that skipped the "berry eaten" announcement had also skipped the record that Harvest, Belch, Recycle, Pickup and Symbiosis all read. One missing call was five broken abilities. **We wrote down all five expected numbers before running, on disk, and all five came out exactly right** - the 961 compared games came back completely unchanged, which we predicted because Cheek Pouch appears in only 10 of the 13,214 games in our recorded pool and Ripen in 2. That is a real fact about the metagame, not a failed fix: the test board moves, and the ladder does not have these Pokemon on it. **One thing we found and left alone on purpose**: Ripen is supposed to halve a softened attack a SECOND time, and it needs exactly the announcement we just built, so it is now possible for the first time - filed as its own job rather than bundled in.

5.230.0 - TWO THINGS ANNOUNCED THEMSELVES AT THE WRONG MOMENT, AND WE CHECKED WHETHER THEY WERE ONE PROBLEM INSTEAD OF ASSUMING IT. Mimikyu takes one hit for free and then its disguise breaks. In the real game the break is ANNOUNCED at the very end of the attack - after the other Pokemon the same move hit has taken its damage, after any stat drop the move causes. Ours announced it immediately, so on a move that hits two Pokemon the break was reported before the second one had been hit at all. Separately, Cheek Pouch gives a Pokemon extra healing for eating a berry; the real game eats the berry, applies the berry's own healing, and THEN adds the ability's. Ours added the ability's first, and the berry's healing line then reported the combined total as if the berry had done all of it. Both are fixed. **They are two separate problems, not one, and we proved it with a 2x2 rather than reasoning about it.** We also checked whether the disguise one was a known open bug we already had on the books - it is not; that one is about a different question and was closed a week ago. Seven of the 961 compared games stopped disagreeing and the total fell from 181 to 175. **We wrote down all four expected numbers before running, on disk, and all four came out exactly right.** No health total changed, no game changed hands, and we said in advance they would not. While reading the code for this we found a related gap - a berry that softens an attack does not trigger Cheek Pouch at all - proved it on a test board, and filed it for its own turn rather than bundling it in.

5.229.0 - TWO THINGS WERE HAPPENING AT THE WRONG MOMENT INSIDE A SINGLE ATTACK. When a move hits twice, the real game runs the whole hit - the damage, and then whatever the defender does back - before starting the second hit. So a Pokemon with spiky skin hurts you after the first hit AND after the second, in between them. Ours dealt both hits and then paid both punishments at the end, which is the same total in the wrong order. Separately, a berry that softens one type of attack is eaten while the game is WORKING OUT the damage, not while it is applying it - which matters when a move hits two Pokemon at once, because then every berry is eaten before anybody takes damage. Ours ate it in the middle. Both are fixed. **We expected these to look like one problem and checked instead of assuming: they are two, and the berry one was not about multi-hit moves at all - it shows up on a plain attack that hits two targets.** Twelve of the 961 compared games stopped disagreeing and the total dropped from 191 to 181, close to the 183 we wrote down before running it. No damage number changed and no game changed hands, and we said in advance that they would not. One of our own tests had a note saying this could not be fixed without rebuilding the whole attack loop; it could, and the note is now a working check instead of an excuse.

5.228.0 - TWO END-OF-TURN EFFECTS WERE HAPPENING IN THE WRONG ORDER, AND ONE OF THEM WAS NOT IN THE QUEUE AT ALL. At the end of every turn the real game runs a fixed list of small effects - the poison tick, the burn tick, the healing berry, the countdown timers - in a published order. Two of ours were out of place. A BURN comes one slot AFTER a poison in the real game, so on a board with both, every poisoned Pokemon takes its damage before any burned one does; ours ran all of them together in speed order and mixed them up. And PERISH SONG's countdown, the three-turn timer that knocks a Pokemon out, was not in the list at all - it ran after everything else, so a Tailwind running out was announced before the countdown ticked. Both are fixed to the order the real game publishes, and neither

needed a single new number: the file that carries those orders already had them right. **Ten of the 961 compared games stopped disagreeing about ordering, and the total dropped from 199 to 191 - exactly what we predicted before running it.** Nothing about the final result of any game changed, and we said in advance that it would not.

5.227.0 - WE WERE HANDING OUT A REWARD FOR A KNOCKOUT AFTER THE GAME WAS ALREADY OVER, AND PAYING IT TWICE WHEN IT SHOULD BE PAID ONCE. Some Pokemon get stronger every time they knock something out. In the real game that reward is handed out once, after all the knockouts on that turn have been announced, and its size is the number of Pokemon that fell - so taking out two at once is one reward of two, not two rewards of one. And if that knockout emptied the opponent's team, the real game simply stops and the reward is never handed out at all. Our simulator did none of that: it paid as each body fell, one at a time, and it kept paying after the game had ended. Fixed, and checked against the real game on a board built so that the only difference between the two turns is whether the opponent still had a Pokemon left. The control that matters is the opposite one - a knockout that does NOT end the game must STILL pay - so a simulator that simply learned to stop after any knockout would fail rather than pass. **Four of the 961 compared games stopped disagreeing, and five separate disagreements were removed with none added.** That was more than predicted: we had said none would move, because we had forgotten our own third point - a reward handed out after the final bell is still sitting on the board when the game is scored. We also found that two of our own checking tools were looking for the wrong wording and briefly reported a correct fix as a broken feature.

5.226.0 - A MOVE THAT HITS SEVERAL TIMES STOPS THE MOMENT IT KILLS, AND OUR SIMULATOR KEPT CHARGING FOR THE HITS THAT NEVER HAPPENED. Some attacks hit two or three times in a row. If the target dies on the first hit, the real game simply stops - there is nothing left to hit. Some Pokémon punish you for touching them, and our simulator was charging that punishment once for every hit the attack WOULD have made, not once for every hit it actually landed. So an attacker lost twice the health it should have, on a turn our own scoreboard was already reporting as one hit. Fixed, and checked against the real game on the same board: the real game charges once, we now charge once, and the control - the same attack into something that survives both hits - still charges twice on both. **Two of the 961 compared games stopped disagreeing, which is exactly what was predicted before the run.** We also re-read the batch of problems this came from and found the grouping was wrong: three of them were filed as one family and are really three separate things, and one of the three was described backwards - the effect fires the right number of times, it just prints in the wrong place.

5.225.0 - WE FOUND EVERY PLACE THE SIMULATOR ASSUMES "THE OTHER SIDE" MEANS "THE OPPONENT", AND WROTE DOWN WHICH ONES ARE RIGHT. In a doubles game you are allowed to point a move at your own partner. Six separate bugs in two days all came from the same silent assumption that you never would. This pass went through all twenty-two places in the simulator that pick a half of the field and recorded, for each one, whether it is genuinely asking "who are my opponents" (seven of them, all correct) or "where is the thing I am aiming at" (fifteen, of which five were wrong). Two of the five are now fixed: a move that blows your own partner off the field - Roar, or Dragon Tail - did nothing at all in our simulator, because it went looking for your partner among the opponents, did not find them, and gave up without saying so. The real game drags them out. Three more are written down with the exact line of the real game's code that proves them wrong, and are handed on as separate jobs. We also left behind a scanner that lists every such place in the simulator and refuses to let a new one appear without somebody saying which question it answers - and it already catches four ways of writing it that a simple search misses. As predicted before the run, the 961-game comparison against the real game did not move at all: nobody in a real ladder game ever aims one of these at their own partner, so this was lab work by construction.

5.224.0 - WHEN YOU ORDER A POKEMON TO ATTACK AGAIN, IT ATTACKS THE SAME SLOT AGAIN. The real game remembers WHERE the first attack was pointed and sends the repeat back to exactly that spot. Our simulator forgot, and chose again — it picked whichever opponent the attack would hurt most. So the free second attack kept landing on the wrong Pokemon, sometimes into a shield that was never in the way. Worse, for a status move like a stat drop it had nothing to compare and simply always hit the opponent on the left. Fixed, and measured on 961 real ladder boards: the games where our simulator and the real one disagreed about the board went from 91 to 90, and turning the fix back off reproduces the old number exactly, so nothing else moved. The board it fixes is a real one: a Gengar aimed a ghost attack at an Oranguru that cannot be hurt by ghost attacks at all, was ordered to repeat it, and our simulator sent the repeat into the Pokemon beside it instead — because a "pick the best target" rule will never aim at something it cannot hurt, and the real game happily does.

5.223.0 - IF A POKEMON IS HIDING BEHIND PROTECT, YOU CANNOT ORDER IT TO ATTACK AGAIN. One move in this format tells an opposing or friendly Pokemon to repeat what it just did — a free extra attack, often the biggest one on the board. Our simulator handed that free attack out even when the target was behind a shield, which the real game refuses. That is a whole extra turn of damage that never happens. Fixed, and measured on 961 real ladder boards: the games where our simulator and the real one disagreed about the board went from 92 to 91, and the single cause that changed is exactly this one. While testing it we found a second, unrelated bug — the repeated attack aims at the wrong Pokemon — which is written down and left for its own pass rather than smuggled in here.

5.222.0 - THREE OF FIVE ALARMS WERE THE ALARM, NOT THE FIRE. Five of our own tests had been reporting a problem for days. We looked at all five properly. Three of them were never looking at the game at all.

One test was being killed for running out of memory before it could finish. It says in its own file how much memory it needs, and our main test runner reads that and gives it. But the shortcut command we tell everyone to use for big jobs did not read it, so the test died every time and the crash got written down as "the game engine got the turn order wrong". Given the memory it asked for, it passes everything.

A second test was writing a results file, and we have a safety catch that refuses to overwrite a big result with a small one. That catch was doing its job perfectly. The side effect was that the test reported failure every single time, no matter how well the game engine was working. There was nothing to fix in the engine, because nothing in the engine could have changed the answer.

A third test had an expectation written into it that was simply wrong. It insisted that a particular control case should stay silent, when the correct behaviour for that case is to speak up - and the test's own printout, two columns to the left of the complaint, showed that the engine had done exactly the right thing.

We did find one genuine problem, and we wrote it down instead of fixing it. A move that lets a teammate or an opponent act a second time does not check whether that Pokemon is hiding behind a shield. So it hands out an extra turn that should not exist. That is a real difference in the game, not just in the wording. We left it alone on purpose: other people were changing the engine at the same time, and a fix made now could not be told apart from theirs.

Nothing we have published changed. No result was updated, no number moved and no results file was rewritten.

5.221.0 - A MOVE THAT INSULTS YOUR POKEMON AND RUNS AWAY ONLY RUNS AWAY IF THE INSULT LANDED.

There is a move in this format that lowers two of your Pokemon's attack stats and then swaps its own user out for a teammate. Two effects in one click, and it is one of the most popular moves in the game.

What we had not noticed is that the second half depends on the first. In the real game, if nothing actually happened to your Pokemon - because it has an ability that refuses stat drops, or because those stats are already as low as they can possibly go - then the user does NOT get to leave. It just stands there. There is exactly one exception, an ability that reflects the drop back at the attacker instead of blocking it, and against that one the user does leave.

Our copy always swapped. So we happened to get the exception right, but only because we got everything right in the same wrong way, and everywhere else we handed the opponent a free escape the real game refuses. The result is the worst kind: the wrong Pokemon is standing on the field, and it stays wrong for the rest of the battle.

We also went and asked the game itself which abilities can do this, instead of trusting the note someone wrote. There were three, not two - the extra one protects a Grass-type TEAMMATE rather than itself. A fourth blocks only one of the two stats, and because the other one still drops, the user DOES get to leave; that one is now a test that has to keep passing, because a careless fix would have broken it. A fifth is named in the real game's code and no Pokemon in this format has it, so we left it alone.

Across 961 recorded games, the number where our board ever differs from the real game went from 93 to 92, and the narration disagreements from 208 to 207. We had said we expected NO change in that figure, because this situation is rare in real play. It changed by one. We were wrong, and we are saying so rather than rewriting what we predicted.

5.220.0 - SOME ABILITIES MAKE A MOVE GO FIRST. WE REMEMBERED THAT WHEN WORKING OUT THE ORDER, AND FORGOT IT EVERYWHERE ELSE.

A handful of things in this format exist to stop a move that goes first. Two abilities refuse one aimed at their own side. A one-turn shield blocks them for the whole team. One move only works if its target is about to use one. A field effect stops them outright.

All five of those need to know one thing: is this move going first? And some abilities MAKE a move go first that normally would not. We handled that when deciding who moves first - and then handed all five of those refusals the move's ordinary speed instead, as if the ability were not there.

So an attack that the real game refuses went through in our copy, and the Pokemon that should have been protected took the hit. That is the worst kind of difference to have: a Pokemon that lives in one version of the battle and faints in the other.

There is now one place that answers "how fast is this move", and everything asks it.

Across 961 recorded games, the number where our board ever differs from the real game went from 94 to 93, and the narration disagreements from 211 to 208. We said before running it that we expected about one game, and that is what it was.

5.219.0 - IF SOMETHING FORCES YOUR POKEMON TO USE A MOVE, WE HAD TO WORK OUT WHO IT WAS AIMED AT - AND WE ALWAYS PICKED THE OTHER TEAM.

Some moves are aimed at your own partner. Helping Hand boosts your partner's attack; Coaching gives your partner +1 attack and +1 defence. Normally you click them and you say who they are for, so there is nothing to work out.

But some things take the choice away from you. Encore forces you to repeat your last move. Copycat makes you use whatever move was used just before yours. Nobody named a target in those cases, and the real game works it out from the move itself: a partner move goes to your partner, full stop.

We were sending it at the other team instead. The +1/+1 from Coaching landed on an opponent. And a forced Helping Hand aimed across the field ran into an ability that stops fast moves coming at its own side - so our version showed the move being blocked, when in the real game it was never pointed that way at all.

The ability was doing the right thing with the wrong information, and that is worth saying plainly: it looked like the ability was broken and it was not. We checked by playing an ordinary clicked Helping Hand at a partner on the OLD code, and it worked perfectly, exactly as it always had.

This did not change who wins any of the 961 recorded games we test against. We said so before we ran it, and that is the answer we expected: both of the things it fixed were commentary rather than anything that moves a board.

5.218.0 - PROTECT GETS HARDER EVERY TIME YOU USE IT IN A ROW. WE WERE NOT COUNTING IT WHEN SOMETHING ELSE MADE YOU USE IT.

Protect works the first time. Use it again the next turn and it only works one time in three; again after that and it is one in nine. The game keeps a counter for that, and our counter was right - we checked it nine different ways and it matched the real game every time.

The problem was one step earlier. We decided at the START of a turn whether a body was putting up a shield, based on what its player had clicked. But two things can change what you use AFTER you have clicked: Encore, which forces you to repeat your last move, and Instruct, which makes you use your last move a second time. When either of those handed a body a Protect, we never ran the shield check at all. So we announced it from whatever was left over: a failure if the body had no shield up, and a FREE shield - one that never had to pass the one-in-three - if it did.

Over 961 real games that was 97 games with a different board, and it is now 94. The counter itself shows up on eleven boards instead of thirteen. Instruct turned out to be the same bug as Encore rather than a second one, which we established by testing them separately.

What is left is smaller and different: four games where both engines agree the odds are one in three and disagree about the coin. One of those four now goes our way instead of theirs, which is how you can tell it is the coin and not the rule. That is written down for next time.

5.217.0 - ENCORE DOES NOT JUST CHANGE WHICH MOVE YOU USE. IN THIS FORMAT IT CHANGES WHEN YOU USE IT.

Encore forces a Pokemon to repeat its last move. If it lands on somebody who has already picked something for this turn but has not moved yet, this format's version of Encore reaches into the turn order and moves them - because the move they are now forced into may be a faster-priority move than the one they chose. Ordinary Pokemon does not do this; the version everyone reads about leaves you where you were. We had implemented the ordinary version.

So a body Encored into Protect, or Helping Hand, or Rage Powder went at the end of the turn instead of near the front. In one real game that meant its Protect failed, because a Protect that is the last thing to happen in a turn has nothing left to protect against.

Nine of the eleven turn-order disagreements our comparison found across 961 real games were this one thing. Fixing it did not change how many games end up with a different board, because those games were going to differ anyway - what it did was uncover WHY. Two new problems became visible underneath it, both written down.

We also found a smaller thing in the same place: when a move gets overridden we were pricing its priority off one move and its "is this a status move" flag off another. The real game reads both off the same record.

5.216.0 - SAFEGUARD PROTECTED US FROM THEM AND NOT FROM US.

Safeguard puts a five-turn bubble over your side that refuses status - paralysis, burn, sleep, confusion. Our simulator only checked the bubble when the status came from the opponent. If your OWN partner paralysed you - which happens, because several moves that hit everything next to them also carry a status, and because people do aim things at their own side on purpose - the bubble did nothing.

The real game does not work that way. Its rule is "not from yourself", not "not from your side". The only place the real rule mentions sides at all is in a clause about an ability that sees THROUGH the bubble, and it is written so that ability cannot see through it for a teammate.

One line came out. Because everything that asks "does the bubble refuse this" goes through a single place, that fixed the status half, the confusion half and the message underneath it at once. We staged eleven checks around it, seven of which exist to prove we did not over-correct: you can still put yourself to sleep with Rest under your own Safeguard, their Safeguard still does not cover you, and it still does nothing about damage or about a Speed drop.

Two more mechanics are now proved. Across 961 real ladder games nothing moved, which is what we said would happen before we ran it - Safeguard shows up in 17 games out of 13,214.

5.215.0 — WE WERE TICKING OFF "PROTECT WORKS" ON A TURN WHERE NOBODY ATTACKED IT.

Our test harness plays a real game for every move in the format and asks the official simulator whether the move did anything. For Protect, Detect, Spiky Shield, King's Shield, Baneful Bunker, Quick Guard, Wide Guard and Endure, the answer was yes — because putting the shield up prints a line, and a line counts. But the opponent in those games was clicking Agility. Nothing was ever thrown at the shield. We had proved the shield goes up and never once proved it blocks.

The harness now asks the second question too, and it works out what to look for by reading the official simulator's own code rather than by us writing it down. Seven of the eight guards now block something for real, and both engines agree on every one. Endure still cannot be tested, and the harness says so out loud instead of passing: our test bodies have six times the normal health so that nothing dies mid-test, which means no hit is ever lethal, which is the only thing Endure reacts to.

Nothing broke and no engine bug came out of it. What changed is that a green tick which meant "we didn't ask" now says so.

5.214.0 — ONE MOVE IGNORED THE POKEMON STANDING IN FRONT OF IT.

In doubles, some moves and abilities exist to say *hit me instead*. Follow Me and Rage Powder do it; the whole point of them is to take a hit meant for a partner. Our simulator obeyed that for attacks, and it obeyed it for status moves like Will-O-Wisp. It ignored it for exactly one move: Parting Shot, which lowers the target's Attack and Special Attack and then switches its user out.

The reason is not interesting and that is the point. Inside the engine, a move that swaps its user out is labelled the same way an ordinary switch is, and the redirection code skipped anything wearing that label. Parting Shot is the only move in this format that was caught by it — and it is one of the most-clicked moves there is.

So in seven of the 961 test games, the wrong Pokemon walked away with the stat drops, and it stayed wrong for the rest of the battle. Fixing it takes the number of games where our simulator ends up with a different board from the real game from **114 down to 106**, and the count of game mechanics we can prove we play correctly from **786 to 788**.

Two related things in the same review turned out to be different problems, not this one, and we said so instead of claiming them: an ability that is supposed to pull your OWN partner's electric move onto itself, and Wide Guard failing to protect a partner from your own side's spread move. Both are reproduced and both are written down. Neither is fixed yet.

5.213.0 — OUR SIMULATOR FORGOT THAT A POKEMON GETS HEAVIER WHEN IT MEGA-EVOLVES.

Four moves in this format hit harder the heavier something is. Our simulator wrote down how much each Pokemon weighed when the battle was set up, and then never changed it. The real game does change it — a Pokemon that mega-evolves is a different Pokemon, with a different weight. So we were still using the old weight, and those four moves came out at the wrong power.

The clearest example: Grass Knot into a Staraptor that had just mega-evolved. The real game deals 23. We dealt 11 — less than half — because we were still weighing the Pokemon it used to be. Two Pokemon of that weight fall into two different power brackets, and we were reading the wrong one.

We fixed it in the same place the real game does: whenever a Pokemon changes into a different form, its weight changes with it. Games where our simulator and the official one end up with a different board fell from 117 to 114 out of 961, and our test suite went from 784 to 786 checked mechanics with none missing.

5.212.0 — WE WERE MARKING GAMES AS "THE TWO ENGINES DISAGREED" WHEN THE ONLY DISAGREEMENT WAS OUR OWN TEST RIG.

Our simulator plays a whole turn in one go. The official simulator we check ourselves against stops in the middle of a turn whenever a Pokémon uses a move that switches it out, and asks who is coming in. Our test rig answered that mid-turn question by looking at who was standing at the END of the turn — and sometimes that is a different Pokémon, because the one that came in got knocked out and the original walked back. So the rig named a Pokémon the official simulator already had on the field, got refused, and wrote the game down as a disagreement. The two engines had agreed on every single line.

Nothing about how we play Pokémon changed. What changed is that 42 of 961 test games were being cut short for that reason, and now 27 are — and the 27 left are real. Games we play to a finish went from 47.8% to 48.4%, and games where the two engines genuinely end up with a different board went from 135 to 117. Eighteen of the disagreements we had listed were never disagreements.

5.211.0 — THREE THINGS OUR OWN SCOREBOARD DOES NOT COVER, NOW PRINTED BESIDE IT.

Nothing we have claimed changed. Three of the claims are now narrower on the page than they looked. First, the stat spreads in our biggest test are INVENTED — a public team sheet does not show them, so we make them up from a fixed recipe and give the same made-up numbers to both sides. That is a fair fight, and it is not the damage a real ladder game does. Second, we compare 34 of the 80 kinds of lingering effect a move can leave behind, and 24 of the other 46 are gone by the moment we look, so the real target is 56 and not 80. Third, a driver that hunts for rare mechanics finishes 2% of its games, while a driver that plays like a person finishes 48% — and the clean sheet we have been quoting came from the first one. Those are two different tests, not a before and after.

5.210.0 — WE PULLED A NUMBER THAT HAD NOTHING BEHIND IT.

A comparison we had been quoting for a month said our win-probability model was BEATEN by a two-number rule of thumb. The script that produced it prints its answer to the screen and saves nothing, so there was no file anyone could open to check it — and it was run before we started throwing out bot and forfeit games, so it was scored on the wrong pile. Run properly, the model and the rule of thumb TIE. The old comparison is withdrawn and left in the review that made it, not deleted. This page never quoted it.

5.209.0 — AN ABILITY THAT EATS FIRE WAS NOT GETTING ANYTHING FOR IT.

The 5.208.0 page below is left as it was written; its numbers come from the older copy of the simulator.

What we fixed. Flash Fire lets a Pokemon swallow a Fire attack whole and then throw Fire attacks of its own about half again as hard for the rest of the battle. Our simulator got the swallowing right and then quietly threw the reward away, so the Pokemon hit exactly as softly as before. It also said "it had no effect" the first

time it ate a Fire move, where the real game only says that the SECOND time — the first time it says "Flash Fire started". Both halves are the same one line in the real game, so one change fixed both.

How we checked. We played the real game, on the same Pokemon, with one thing changed: whether the attack that came in on turn one was a Fire attack. The follow-up attack dealt 91 damage when it was not and 136 when it was. Ours now does the same thing for the same reason.

And we made the checker look at one more thing. When a Pokemon holds a Choice item it is locked into the move it used first. Our board comparison was not looking at that lock at all, which means it could not have caught us getting it wrong. It looks now — at WHICH move, not just at whether there is a lock — and across 12,445 checkpoints of 961 games the two simulators agreed every time.

Version 5.208.0 · 2026-08-28 · Will Hooper

5.208.0 — ONE MORE ITEM NOW WORKS, AND WE CHECKED EVERYTHING AGAIN AFTERWARDS.

The 5.207.0 page below describes the day we accepted the last disagreement. It is left as it was written, and the numbers on it are from the older copy of the simulator.

What we fixed. An item called Metronome makes an attack hit harder every time you use the same attack twice in a row, up to a limit. We had worked out exactly how that ladder behaves nearly three weeks ago, written it down correctly, and then never connected it to anything. The simulator was ignoring a rule it already knew. It does not any more.

Why we re-checked everything. Changing the simulator means every earlier comparison was made against a different program. So all five comparisons were run again from scratch, on the new copy. That is the rule here: a result belongs to the exact version of the code it was measured on.

What the re-run says. Every item, ability and move staged one at a time: **140, 129 and 475 tested, nothing disagreeing and nothing failing to happen**. Whole real games played side by side against the official engine: **961 games, 6 differences, all six already declared and explained**, and every one of the 12,445 end-of-turn boards identical. The full sweep of game mechanics: **782 checked, 782 working, none missing** — two more than before, because the fix added two new things to check.

The scorecard is now clean: eight boxes of eight. That is the first time all eight have been green.

What that does not mean. About seventy older results stopped being held back — but they are not suddenly true. They were measured on an older simulator, and every one still has to be run again before anyone quotes it. Not one of them was re-run. And the one message-ordering fault we accepted last time is still on the list as a fault we chose not to fix.

One honest note. Metronome is a rare item — nineteen teams out of twenty-six thousand in our frozen sample carry it. We said before the run that the real-games comparison would not move, and it did not. The staged laboratory saw the fix; the real-games sample correctly saw nothing. That is one instrument confirming it, not two.

5.207.0 — THE LAST DISAGREEMENT IS ACCEPTED ON PURPOSE, AND IT IS WRITTEN DOWN IN A WAY THE CHECK CAN READ.

The 5.206.0 page below still describes the day the five boxes went blank. It is left as it was written.

Where we got to. We play 961 real games through our simulator and through the official one, with every dice roll forced to match, and compare them line by line. Six games came out different. Five are a typo in the *official* simulator — it prints the word `fallenundefined` on a line nobody ever sees — and copying a typo is not being correct. The sixth is one game where a Pokémon dies to Perish Song and we announce the death one line earlier than the official simulator does.

Nothing about the board is different. We checked the thing that would show it: every Pokémon's HP and whether it is fainted, at the end of every turn, in 12,445 turn boundaries. All 12,445 match. The Gengar is dead in both, at the same HP. Only the *position of the message* differs.

Will's call: accept it and move on. So it goes in the closet. The important part is that the closet is a mechanism and not a sentence: the entry has to name who ruled it, on what date, in what words, what instrument measured the "no board effect" claim, which frozen version of the engine that measurement was taken on, and exactly what observation would prove the entry wrong. If any of those is missing the check refuses the entry and the gate stays shut. **We have not called it fixed and we have not called it harmless — we have called it a defect we chose not to fix, which is what it is.**

One thing this does not mean. The gate opening releases about sixty older results that were being held back. Those results are not suddenly true. They were measured on an older engine and every one still has to be re-run before anybody quotes it.

5.206.0 — THE FIVE BLANK BOXES ARE FILLED IN, AND EVERY ONE OF THEM CAME BACK WITH THE SAME NUMBER IT HAD BEFORE.

The 5.205.0 page below still describes the day the boxes went blank. It is left as it was written. What it says about *five blank boxes* is now out of date, and here is why.

What was actually wrong. Nothing about the game. One data file that the project generates was saved by a routine git step with different invisible end-of-line characters — same content, same meaning, more bytes. The project identifies "which exact copy of the simulator did we measure?" by the bytes, so a file that gained invisible characters looked like a different simulator. Five measurements were taken against the old copy, so the project refused to show them.

What we did. We put the file back the way the program that generates it writes it — by asking git for the version it already had stored, not by editing it — and then we ran all five measurements again from scratch. Then we stopped it happening a third time: git is now told, in writing, never to translate line endings on those files. We proved that instruction works by making the fault happen on purpose first, watching it fail, adding the line, and watching it stop.

What the five measurements say.

- The three "every item / every ability / every move, staged one at a time" sweeps: **139, 129 and 475 tested, with nothing disagreeing and nothing failing to happen** on any of them.
- Whole real games played side by side against the official engine: **1 game out of 961 differs**, and **zero** of them differ in a way that would change a decision.
- Every mechanic staged and compared: **nothing counted against us.**

The honest part. Re-running produced numbers identical to the ones we had withheld — the game files differ only in how many seconds each run took. That is a good outcome and it is also a small one: it means the withholding was correct and cost us nothing but time. **The scorecard is still not clean.** One box of eight is still red, it is the same one it was, and the simulator is still not signed off.

5.205.0 — WE PAUSED THE BIG SIMULATOR PUSH. HERE IS WHAT WE FIXED, AND — MORE IMPORTANTLY — WHAT WE STILL CANNOT TELL YOU.

For eighteen days we stopped updating these documents on purpose, so that fixing the simulator did not have to stop every hour to write about itself. Instead every fix wrote one line to a running log. That log had 274 lines in it when we stopped. The changelog has 233 releases from the same stretch, and there are 189 dated write-ups in the reports folder. The log is now deleted and the normal write-it-up-as-you-go rule is switched back on, which is the whole point of deleting it.

THE MOST USEFUL THING WE LEARNED IS NOT ANY ONE FIX. It is that the thing doing the measuring was wrong at least six times. Six separate occasions where we thought we had found a broken simulator and had actually found a broken ruler. One night we had thirty accusations against the engine; when we checked them properly, twenty-three were the test being written wrong, seven were the rule being written wrong, and exactly **one** was the game actually being simulated wrong. Another time, eighteen out-of-date certificates were being published as though they were a broken simulator. If you take one thing from this page: **before you believe the scoreboard, check the scoreboard.**

AND THE BIGGEST FIX MADE THE PROBLEM LOOK WORSE, WHICH IS EXACTLY WHAT IT SHOULD HAVE DONE. Deep inside the simulator is a dice roller. It is not real randomness — it is deliberately repeatable, so that we can play the same game twice and compare. The way it turned a situation into a number had a flaw: if two events differed only in "which hit of the move is this", it did not really roll again. It nudged the previous answer slightly. Two hits in a row landed in the same damage bucket **89.5%** of the time, when the honest number is about **6%**. The roller looked perfectly fair if you averaged it over a whole run, and was almost completely predictable one step at a time.

Because of that, our simulator and the real game had been agreeing partly by luck. They were only ever being compared across a thin slice of the things that can happen. We fixed the roller and the number of games where the two disagree went from **3 to 14**. Nothing got worse. We could finally see.

SO: EVERY NUMBER IN THIS DECK THAT WAS MEASURED BEFORE 27 AUGUST 2026 IS DEAD, NOT MERELY OLD. Not "a bit out of date" — genuinely not evidence of anything, because the comparison behind it was narrower than it claimed. We leave the old paragraphs in place, because this project does not quietly delete things it used to believe, but **none of them is a current result.** If you want to know where something stands today, run `node engine/status.js`. If that prints nothing for it, we do not know.

WHAT WE CAN TELL YOU TODAY

- **Every mechanic we have thought to test is tested, and they all fire.** 780 checks, 780 of them live, none missing. That is a laboratory result: we stage each thing deliberately, one at a time. It tells you *is this right*. It does not tell you *does this come up in real games* — that is a different scoreboard, and it is the one that is currently blank.
- **Damage is exact.** Six thousand comparisons against the real game, zero disagreements, and zero at each of the sixteen points of the damage range checked separately rather than lumped together. **But read the small print, because it is genuinely small:** that test covers damage only — no items, no abilities, no turn order, no switching — and it has **never once tested a move that hits more than once.** It skips all of those. Four multi-hit bugs were found and fixed this month and this test could not have seen any of them.

WHAT WE ARE REFUSING TO TELL YOU, AND WHY THAT IS THE HONEST ANSWER

Five of the eight boxes on our "is the simulator right yet" scorecard are **blank**. Not failing — blank. The measurements behind them were taken against a slightly different copy of the engine than the one now on disk, so those numbers describe a program that is not the one we are running. We could print them with a warning attached. We are not going to, because we have done that before and people quoted the number and skipped the warning — including us.

For the record, the difference is not a real change to the game. One data file got rewritten by a routine housekeeping step with different invisible end-of-line characters. Same content, different fingerprint. It still counts, and the only cure is to run the comparison again.

"THE BOARDS MATCH" MEANS 33 THINGS OUT OF 80. When we say our simulator's game state agrees with the real one, we are comparing 33 kinds of state. There are 80 kinds that a legal move, ability or item can create. Four more are openly declared as not compared. That leaves **43 that are in neither list** —

not compared, and not admitted to be uncompared — and 25 of those can be sitting on the board at the exact moment we take the comparison. A thing you forgot to compare looks precisely like a thing that agreed. This is the biggest caveat on the page.

AND THE FREEZE IS STILL ON. Everything that reads the simulator — how good our position-scorer is, the search results, the head-to-heads, the trained weights — remains withheld. The rule we hold ourselves to is: those numbers do not become true when the simulator becomes correct. They become **re-runnable**. Somebody has to run them again.

3.98.0 — QUICK GUARD IS SUPPOSED TO STOP FAST MOVES. OURS STOPPED NOTHING AT ALL. Some moves always go first — Fake Out, Bullet Punch, Aqua Jet. Quick Guard is the answer to them: it protects your whole side from anything that jumps the queue. We checked every way the game has of stopping a fast move, and four out of five worked. The fifth was Quick Guard, and it did not exist as far as our simulator was concerned — clicking it wasted your turn. The reason is small and embarrassing: Quick Guard and Wide Guard are described in our data with the *exact same four labels*, so the program was telling them apart by their names, which is the one thing we have a rule against. The information that separates them — one stops fast moves, the other stops moves that hit everybody — was already sitting in the data file and nothing was reading it. It reads it now. Quick Guard also correctly stops a status move that a Prankster ability has made fast, and Feint still punches through it, because Feint punches through every shield in the real game. Wide Guard was checked and is unchanged.

3.97.0 — FOUR MOVES DEALT THE WRONG DAMAGE BECAUSE WE ROLLED THEM ONCE AND MULTIPLIED. A move that hits three times should roll three times. Ours rolled once and multiplied the answer, which is fine when every hit is the same and wrong when it is not. Triple Axel gets stronger with each hit — we gave it the weakest hit three times, so it did **exactly half** its real damage. Dragon Darts fires one dart at each opponent — we fired both at one, so the partner took **nothing**. Beat Up is one punch per healthy team-mate, and we added their strengths together into a single punch. Fickle Beam has a 30% chance to go all out, and we gave it 30% of the boost every single time — a number the move never actually has. All four are fixed by giving each hit its own roll. Ordinary moves are untouched, and that was checked rather than assumed: every one of the 500 moves was played through a real turn before and after, and only these four came out different.

3.96.0 — THREE ITEMS DID NOTHING IN OUR SIMULATOR, AND NONE OF THEM WAS ACTUALLY MISSING. Iron Ball halves your Speed. Our engine handles that perfectly — for Choice Scarf. The rule that decides which items get the treatment was written as "the item called Choice Scarf" rather than "any item that does this", so Iron Ball was left out for 139 recorded uses while the working code sat there unused. Same story for Light Ball: the rule listed four items by name, and **all four are banned in this format** — so it could never do anything for anyone. And Oran Berry heals a flat 10 health, while we only knew how to read heals written as a fraction, so we read "unknown" and did nothing — which is the right call, guessing would be worse. All three now work it out from the game's own rules instead of a list of names. **The lesson:** something can be built, correct, and working, and still be switched off for a particular Pokémon or item because a list somewhere never mentioned them.

3.95.0 — WE THOUGHT WE COULD KNOCK OUT A MIMIKYU. WE CANNOT. Mimikyu has an ability that soaks up the first hit it takes — the attack does nothing at all. Our simulator has two halves: one plays out the turn, the other just works out "how hard would this hit". The turn-playing half knew about it and has for a while. The damage half did not, so it kept saying Wood Hammer does 120 to a Mimikyu when the real answer is zero. Anything picking moves was reading the wrong half, so it believed it had a knockout that was not there. Both halves now ask the same single source. **This was a quarter of our "is the simulator correct yet" scorecard, and it was one row.** That section is now clean — 150 out of

1. **The tricky part was the fix, not the finding:** the turn-playing half gets its numbers from the

damage half, so the moment we correctly said "zero", the turn half stopped noticing the ability had been used at all. Caught that before running it.

3.94.0 — A MOVE THAT HURTS YOU TO USE IT WAS FREE FOR US, ON TWO MOVES. Some attacks lower your own stats as the price of using them — Close Combat drops your defences, Draco Meteor drops your special attack. We handle all of those correctly, on more than twenty thousand recorded uses, which is precisely why nobody looked closer. It turns out the real game stores that price in **two different places**, and we only ever read one of them. Two moves keep it in the other place — Clanging Scales and Scale Shot — so for those two the drop simply never happened: the real game lowered the user's defence twice over two turns, ours left it untouched. Fixed by reading both places. **We also raised a much bigger alarm and then shot it down ourselves**, which is worth saying: it looked briefly like a label covering 22,000 uses had nothing reading it, including Close Combat. It does not — the engine gets there another way, and the six moves already testing correct are what proved it. A missing label and a missing mechanic are not the same thing.

3.93.0 — THE MOVES THAT SQUEEZE YOU FOR A FEW TURNS WERE ALL COUNTING A TURN SHORT. Bind, Fire Spin, Infestation, Sand Tomb, Snap Trap, Whirlpool and Wrap trap the target and chip its health for a few turns. All seven were off by exactly the same amount in exactly the same way, which is what told us it was one mistake and not seven. Someone had written down "4-5 turns" — the number a player feels — but the real game keeps a slightly different counter behind the scenes, starting one higher and ticking it down on the very turn the move lands. We now read that counter straight out of the real game instead of writing a number down. This is the third time this exact mix-up has bitten us, and the first two fixes missed this one because it is stored somewhere else in the code. **We proved it was broken before we proved it was fixed**, by running the old frozen copy of the engine and watching it get the wrong answer. Seven test rows went green and nothing else changed. Whole-game agreement did not move, and we are saying so: these moves are rare enough that they hardly ever decide where a game first goes wrong.

3.92.0 — WE WENT LOOKING FOR MORE OF THE SAME, AND TWO OF WHAT WE FOUND WERE REAL. Yesterday we caught a test using Tackle, a move this format does not have, so we checked every test for the same habit. Most were harmless. Two were not. One test needed three Pokémon to stand still and do nothing, and told them to use Splash — a move our simulator has never heard of. They stood still because the simulator could not find the move, not because the move does nothing. That keeps working right up until it does not. The other test only ran a check when the move "exists", and it turns out a banned move still counts as existing — so that check never once did its job, and the case ran on a move nobody in this format can use. Both now use moves that really are in the game, picked from the game's own list rather than typed by hand. No result changed, which is exactly what we expected.

3.91.0 — OUR TESTS COULD TEST THINGS THAT ARE BANNED, AND WOULD HAVE HAPPILY REPORTED THEY WORKED. When we test one small piece of the game, we build a fake Pokémon and hand it to the official simulator. It turns out the simulator never checks whether that Pokémon is legal. Hand it a banned item and it will play along, our engine will play along, the two will agree, and the test goes green — about something that can never happen in a real match. Will asked the obvious question: Showdown ships a team validator, why aren't we using it? Now we are. It is stricter than a ban list, because it also knows which moves each Pokémon can actually learn — it caught a test we wrote yesterday giving Meganium a Fire move it cannot learn. **We kept one deliberate exception, on purpose.** To compare several abilities fairly you put them all on the same Pokémon, even though that Pokémon could not really have most of them. That is the whole point of a controlled comparison, so it is still allowed — but it now has to be written down as a choice rather than happening by accident. Banned things are never allowed, no exceptions. **The first thing the new check caught was our own test file:** it had been padding empty slots with Tackle, and Tackle is not in this format.

3.90.0 — MOVES THAT HIT TWO TO FIVE TIMES ALWAYS HIT 3.1 TIMES, WHICH IS NOT A THING THAT CAN HAPPEN. Rock Blast, Bullet Seed, Icicle Spear and the rest roll for how many times they hit. Our simulator never rolled: it used the average, 3.1, every single time. So half these moves came out too strong and half too weak, and it looked like eleven separate small bugs instead of one. It was one. The simulator now rolls the count the same way the real game does, and the moves that always hit exactly twice — Double Hit, Dual Wingbeat, Twin Beam — were checked to make sure they did not move, because a "fix" that breaks the simple case has broken something else. Three moves in the group are still wrong and each is a different problem: Triple Axel hits harder with each hit, Scale Shot changes your own stats afterwards, and Dragon Darts spreads its two hits across two opponents. Those are named rather than lumped in.

3.89.0 — SOME POKÉMON GET STRONGER WHEN YOU HIT THEM, BUT ONLY IF YOU HIT THEM THE RIGHT WAY. Anger Point maxes out its Attack when it takes a *critical* hit. Justified only reacts to *dark* moves. Weak Armor only to *physical* ones. Our simulator gave all of them their bonus on every single hit, so it thought hitting these Pokémon was far more dangerous than it is. The one member of the group that genuinely has no condition — Stamina, which is also the only one people actually run — was right the whole time, and that is exactly why nobody spotted the other eleven. Separately, four healing moves (Synthesis, Moonlight, Morning Sun and Strength Sap) did **nothing at all**: the Pokémon spent its turn and recovered zero health. In a sandstorm that was worse than doing nothing, because the sand still hurt it. All fixed, with tests that fail if either comes back.

3.88.0 — TWELVE MOVES WERE PRICED OFF GENERIC GEN-9 DATA INSTEAD OF THIS FORMAT'S, AND THE BUILDER THAT FIXED THEM WAS ONE RUN AWAY FROM DELETING TEN SPECIES. Trop Kick read 70 where the format says 85, Mountain Gale 100 against 120 — ours low in all twelve, and MAG's own table had the right numbers the whole time, so the two engines disagreed on every one. Asking what a regeneration WOULD do, before running one, turned up 788 destructive changes waiting in the same builder and a header stamp whose regex had never once matched. `buffsHolderOnHit` also gained its condition by derivation — Anger Point only on a critical hit, Justified only on Dark — but **the engine does not read it yet and nothing behaves differently**, which is said here rather than left to look like a fix.

3.87.0 — ONE POKEMON CARRIES ITS OWN SUNSHINE, AND HALF THE ENGINE COULD NOT SEE IT. Meganium's mega form fights as if the sun were out, even when the sky is clear. Weather Ball is a move that changes type with the weather: Normal in clear skies, Fire in sun. Our damage maths knew about the private sunshine and called the move Fire. The part of the engine that decides whether a move is allowed to connect did not, and called it Normal — and a Normal move does nothing at all to a Ghost. So the mega's signature attack dealt zero. Both halves now ask the same question of the same function. We checked it against the real game first: 0 damage without the mega, 97 with it, and exactly the same 97 under a real sun. Live mechanics went from 325 to 326, and nothing else moved.

3.86.0 — EVERY PUBLISHED ARTIFACT HAS A WRITER NOW, INCLUDING THE ARM MILTANK ACTUALLY RUNS. The tool that answers "is this number still true" could only find a writer by an artifact's literal name beside a write call in two directories — so the mechanics census, the game differential, the interaction matrix and the deliberate roster, which are the four clauses of the MEDICHAM gate, had no row at all. Not ok, not unsafe: absent. The graph goes 115 to 160 artifacts and the unknown set 61 to 16, membership derived through four ranked arms that each record how they matched. UNSAFE rises 13 to 20 because seven artifacts that were always unsafe are now visible; none left the set.

3.85.0 — THE WHOLE SITE WITHHOLDS NOW, AND THE DEPLOYED COPY WAS MISSING THE FILE THAT MAKES IT WORK. Five pages LOADED a quarantined artifact as data instead of quoting its verdict, so the citation checker could not see them at all; seven of the Stadium's fifteen cabinets

now go dark, each keeping its seat and its button and answering with the quarantine instead of a number. The file that drives all of it, `quarantine-data.js`, did not exist under `app/` — which is the copy a visitor loads — so every guard there took the healthy path. That is the same failure as the day before, one directory over.

3.83.0 — FOUR OF THE COMMONEST ABILITIES IN THE GAME HAD NEVER ONCE WORKED, AND THE SIMULATOR WAS RIGHT TO IGNORE THEM. Blaze, Torrent, Overgrow and Swarm all do the same thing: when the Pokémon drops below a third of its health, its attacks of one type hit 50% harder. Our simulator knew the 50%. It did not know "below a third" — that part was written down as an English sentence, and code cannot read a sentence. Faced with a rule it could not check, the simulator did the safe thing and did nothing, which is right: guessing the cut-off would have handed every Charizard a permanent 50% bonus. So the fix was to write the rule down as a number the code can check, taken straight from the official game's own source. The catch was the exact line. "A third of 150" is 50, and if you compute it the sloppy way a computer gets 49.999999999999993 — so a Pokémon on exactly 50 would miss out on a boost it had earned. That single point of health is the one that decides games, so the test now checks precisely there, in both directions, against the official engine.

3.82.0 — THE FIRST ENGINE FIX OF THE QUEUE LANDS: THE VOLATILE DURATION FAMILY, 9,092 USES, AND IT WAS THE PERISH SONG BUG A SECOND TIME. Showdown decrements a volatile's duration inside the Residual event, so one applied on turn N has already spent a turn by the end of it. That defect was documented in this engine for Perish Song, fixed for Perish Song, and left standing for every other duration-bearing volatile. Taunt and Disable now match the official engine; Encore's counter row is gone and only a separate HP row remains. Whole-game board agreement rose 76.9% to 78.9% on a paired differential, the roster's moves queue fell from 52 to 50, and the census did not move. The previously published baseline could not be reproduced because the census digest and the team store had both shifted underneath it — so the run was discarded and re-taken paired. The delta is the measurement.

3.81.0 — THE QUARANTINE REACHED THE BOARD, AND THE CHECKER THAT POLICES IT WAS BLIND TO THE PUBLISHED COPY. Thirteen slots on ABRA WORLD's status board now render as a redaction bar rather than a number, each carrying the artifact, the reason and the command that re-runs it. Two defects in the guard itself were found doing it: its own selftest had gone red — an `all`-stage artifact was matching ANY requested stage name, so "a missing stage must FAIL" stopped being enforced — and its citation walker looked at docs/ and web/ but never at app/, which is the copy a visitor actually loads. Five withheld verdicts were being published from app/ the whole time the check read green.

3.80.0 — THE DELIBERATE ROSTER'S INERT BUCKET COLLAPSED BY 94.1% OF ITS USAGE, AND A FAMILY OF ABILITIES WORTH 8,524 USES TURNS OUT NEVER TO HAVE FIRED. 124 abilities were falling through a catch-all that stages a plain attack, so the condition each one needs was never created and the roster honestly reported INERT — which reads as "nothing to test" when the truth is "never tested". Fifteen new shape rules take that bucket to 59 abilities / 4,261 uses, all 22 ability rules caught their own break, and nothing left in the bucket is above 500 uses. The first real defect it found: Blaze, Torrent, Overgrow and Swarm carry their below-1/3-HP condition as PROSE, and the consumer refuses any condition it cannot evaluate — so the pinch family has never once fired. The roster's artifacts are now written per stage, so the quarantine gate reads measurement rather than absence.

3.79.0 — EVERY FIGURE DOWNSTREAM OF MEDICHAM IS NOW WITHHELD RATHER THAN CAPTIONED, AND THE DELIBERATE ROSTER'S MOVES STAGE RAN FOR THE FIRST TIME. Will's standing call: *"all engines that take medicham's output should be regarded as out of date and we should stop referencing them until medicham is up to date and we can rerun them."* 34 of 114 artifacts are downstream and are no longer printed at all — R1, R2, R3, R4, leaf calibration and the weights among them — because a caption is not a quarantine: `PRE-CHANGE` had been printed beside those numbers for days and they went on being quoted anyway. Membership is derived from the dependency graph, not typed, and the gate

that lifts it is computed from the differential and the roster, where a MISSING stage counts as failing. The roster gained 26 move shape rules and staged all 500 legal moves for the first time, returning a 79-row queue over ~15,000 uses. Its control arm was found to be measuring the CONTROL rather than the subject, which made six ability findings false; **Weather Ball, Sand Rush and Damp are retracted as defects and are correct.**

3.78.0 — THE SHEET'S REAL NATURE NOW REACHES BOTH ENGINES, AND THE TURN-1 NUMBER FELL. THAT IS THE INSTRUMENT GETTING HONEST. The whole-game differential built every body `Serious` while the stored sheet beside it said `Modest`, and with every body flat AND Serious, 326 of 357 species in the format share a Speed with some other species — so the rig MANUFACTURED speed ties and almost never tested a real speed differential. Carrying the declared nature cut the tied groups the resolver has to break from 348,595 to 243,467 over the same 1,998 games, a 30.2% fall. The instrument's own numbers went DOWN, as predicted before the run: the board at the end of turn 1 is identical in 97.4% of games flat against 97.3% natured, games whose board never parted 80.8% against 78.8%, and the median turn of first board divergence one turn earlier at 7. Encore divergences nearly doubled (10 to 19 games), which is what a duration volatile that only bites when turn order does looks like when turn order starts being tested. THE SPREADS REMAIN ABSENT AND ALWAYS WILL BE — a Showdown open team sheet does not reveal them (`"evs": null` on 173,784 of 173,784 stored bodies), so this narrows the declared gap and does not close it. Neither engine is told the other's answer: both are told the nature and each computes, and the alignment assertion still reads 0. It read 21 on the first run and all 21 were Ditto — entry-time Imposter had already transformed the medicham body, and the harness was writing the copied stat line onto Showdown's Ditto before the game began. Census 324 live, 0 missing, unchanged.

3.77.0 — CONFUSION DID NOT EXIST, AND BURN HAD NEVER BEEN ON A BOARD. The confusion volatile was written and never read or ticked, so Hurricane's secondary — 3,779 uses — fell through every branch, and the two berries that clear confusion looked dead because there was nothing to clear. The sleep counter was an ordering bug: the authority runs sleep before flinch, ours ran flinch first, so a body that was asleep AND flinched never ticked and woke a full turn late. Burn, by contrast, is CORRECT and was confirmed rather than changed — but it had never once been staged, because Will-O-Wisp is 85-accurate and the harness pin makes every sub-100 move miss. The freeze timer is correct too; what was missing was the instrument, which carried no freeze counter at all, so the engine's value could drift and no measurement would see it. Census 324 live, 0 missing.

3.77.0 — THE ACROSS-A-SWITCH ARM FOUND A DEFECT ONE DAY OLD THAT FIXING SOMETHING ELSE CREATED. A transform never reverts when the body leaves the field: the authority clears it in `clearVolatile`, and this engine sets the flag and never unsets it. Since the transform also overwrites the body's name, stats, types, moves, boosts and ability, a benched Ditto is PERMANENTLY the thing it copied — so the two engines then choose replacements from benches that no longer describe the same Pokemon, and worse, a Ditto can only ever transform ONCE PER BATTLE, because the guard refuses a second. Re-copying is the entire function of the Pokemon. Imposter first fired the day before, and the out-and-back scenario that exposes this only became expressible hours earlier. The roster's two owed arms — across a switch, and at the exact HP line — are both built, both red-demonstrated, and Speed Boost and Focus Sash both match.

3.77.0 — A STAGED SCENARIO CAN NOW SWITCH, SO A MID-TURN ENTRANT IS EXPRESSIBLE FOR THE FIRST TIME. The scenario driver understood only a move; every other step became a pass, so no staged test could put a body on the field part-way through a turn. That single gap blocked three things at once: Speed Boost's entry gate, which exists only for a body that just switched in; Hunger Switch's flip and Zero to Hero's switch-out transform; and the whole across-a-switch arm of the roster. Four of the six engine defects found the day before were about a MOMENT rather than an effect, and

no scenario without an entrant can express one. Verified end to end: Espathra switches in and reads +0 Speed in both engines on the turn it arrives, then +1 at the end of the next, with all 131 fields identical on both boundaries.

3.77.0 — ALL 316 ABILITIES STAGED DELIBERATELY, AND A FREE +6 ATTACK FELL OUT. Anger Point and Justified are one defect twice: a conditional boost-on-being-hit whose condition is never checked, so Anger Point grants +6 Attack off an ordinary hit where it requires a crit, and Justified grants +1 off a Poison move where it requires Dark. Hustle applies no 1.5x Attack at all. Two facts about the instrument matter as much: Gastro Acid does not suppress an ability here, and since suppression is the ONLY control available to 23 abilities, checking that control against a known-live fixture is what stopped five more from being published as dead for the control's failure rather than their own. And a fact about the regulation rather than the simulator: 113 of 316 legal abilities have NO legal carrier, so the effective roster of this format is about 203.

3.77.0 — FIVE MECHANICS THAT DID NOTHING, AND ONE THAT WAS ALREADY RIGHT. Imposter never transformed Ditto; Hunger Switch never flipped Morpeko; Knock Off took its 1.5x against an item it cannot remove; Fling never became an attack at all, because a base power of 0 made the click fail a `hasPower()` gate; and Roar's phaze branch held a Pokemon-first target, so a phaze after a pivot dragged nobody — the SIXTH site missed by the slot-first sweep, and at priority -6 the worst possible place to hold a body rather than a slot. Mawile's mega ability swap, which had been blamed for a whole family of Attack-stage divergences, WAS ALREADY CORRECT: the scenario was board-identical on its first run, and deleting the swap deliberately parts two fields at once, so the symptoms were real symptoms of a bug this engine does not have. Census 319/319 live, 0 missing; the staged harness now carries 24 scenarios, all clean and all breakable.

3.77.0 — THE TEST ITSELF COULD SEND THE TWO COPIES OF THE GAME DIFFERENT POKEMON. When the test tells both engines to switch, it names the Pokemon it wants. One copy looked that name up one way and the other looked it up another way, and the two only agreed while nothing had been renamed — which stopped being true the day we taught the engine that Mimikyu becomes Mimikyu-Busted and Palafin becomes Palafin-Hero. Worse, when either copy could not find the Pokemon it simply did nothing and said nothing, so one side could switch while the other stood still and the difference looked like an engine bug. Both now ask the same question, and any failure to find it is counted out loud.

THE ENGINE FOLLOWED THE POKEMON, AND THE GAME FOLLOWS THE SPOT IT WAS STANDING IN (3.77.0). If you aim at the left-hand Pokemon and it switches out before your move goes off, your move hits whatever walked in. Our simulator kept aiming at the Pokemon — sometimes at one that was already on the bench — so stat drops from Charm and Parting Shot went nowhere. Ally Switch, which swaps your two Pokemon between spots, did not exist at all: the simulator treated it as a wasted turn. And Speed Boost was handing out its Speed one turn too early to anything that had just come in. All three are fixed, each proved by playing the same turn in both simulators and comparing every field of the board.

A COIN FLIP WE HAD BEEN CALLING THE SAME WAY EVERY TIME (3.74.0). When two Pokemon are exactly as fast as each other, the real game flips a coin. Our simulator did not — it always let the same one go first, and it had done that since the day it was written. That matters more than it sounds: nine out of ten Pokemon in this format share their Speed with some other Pokemon, so this was not a corner case, and it was not just a test problem — it is the same code that decides the order in every game the bot plays. Fixing it was not a matter of flipping the answer round. The official engine sorts in a way that shuffles ties, and the tempting shortcut — "always let the second one go" — would have made our test go green while the bot stayed wrong in real games. We copied the real rule instead, so both engines land on the same Pokemon for the same reason. Alongside it: Palafin was transforming at the wrong moment, Mimikyu's broken disguise

never changed its name on the board, and a Pokemon that switched out mid-attack was paying its own damage costs after it had already left. Twelve more abilities and moves that nobody had ever tested now have tests. Two of them turned out to have been working all along.

WE WERE ONLY EVER TESTING ONE CORNER OF THE GAME (3.73.0). To compare two engines fairly you freeze the luck, so both get identical dice and any difference has to be a real bug. We froze it to a single setting — and that setting meant the speed tie always went the same way, every move under 100% accuracy always MISSED on both sides, and damage was always the maximum roll. Rock Slide had never once connected in the entire history of this test. It does now. There are four frozen settings instead of one, so we look at four corners rather than the same corner four times, and a comparison between runs with different settings is refused rather than quietly reported. We also stopped counting a move as tested just because the bot clicked it: clicking Haze when there are no stat boosts on the board does nothing at all, and the old rule marked Haze covered and moved on. Five mechanics turned out to have been "covered" that way while doing nothing. **Every score published before this is measured on a different set of games and cannot be compared with what comes next.** And one real engine bug came out of it: when two Pokemon have exactly the same Speed, the real game and our copy have been picking different ones to move first — every time, for as long as this test has existed.

ROADMAP #92 — THE DAMAGE-STAGE CLASS. FOURTEEN MULTIPLIERS WERE APPLIED AT THE WRONG STAGE AND FIVE WERE ABSENT (3.73.0). Showdown applies each multiplier at a STAGE — a base power, a stat, or the final damage — folds every handler at that stage into ONE modifier, and spends it ONCE. This engine applied about a third of them a stage late, and separately: Black Glasses on the final damage where the authority puts it on base power reads 109 against 108. That one-point shape is why it survived every existing check — both engines "apply Black Glasses", so the census saw it LIVE, the interaction matrix compares a ratio between arms, and the damage differential allows a 12% midpoint band by design. LANDED: the 18 type items, Muscle Band, Wise Glasses, Technician, Tough Claws, Sharpness, Iron Fist, Mega Launcher, Strong Jaw, Punk Rock, Sheer Force, Supreme Overlord, Expanding Force, Rising Voltage, Dry Skin and the -ate x1.2 into ONE base-power relay spent once; Thick Fat (73% wrong), Heatproof, Purifying Salt and Water Bubble (77%) into the STAT relay, because they modify a stat and not the damage; Helping Hand (wrong on 5 of 5 audited rows) and Friend Guard (21.4%) off the hit site and into the chains they belong in; Sniper out of the crit's plain multiply and into the final chain; and the rolled crit's POSITION into the damage range before the randomizer, where it was 46.5% wrong at the bottom roll and invisible at the top one every check pins. The four FIELD terrains were absent entirely — a Grassy-Terrain Earthquake was priced at DOUBLE the real number. New gate `tests/test-damage-stages.js` is **1,728 of 1,728 exact** against the authority across all sixteen damage rolls and both crit states, and was shown RED on two deliberate reversions before being trusted. `damageBoost` is still NOT wired as a class and the reason is a property of the tag: it carries neither the stage nor the condition, so wiring it would hand Blaze a permanent x1.5 on 5,808 sheets. Census 293/294 → 298/299 live.

ROADMAP #81 WIRE 12 — FIVE ENGINE DEFECTS OFF THE TURN-1 BOARD, TWO OF THEM MIS-DIAGNOSED BEFORE THEY WERE FIXED (3.71.0). The auras (Fairy, Dark, Aura Break) are wired FIELD-WIDE at the base-power stage — they multiply one type for every body on the field, the foe's moves included, and Aura Break INVERTS to x0.75 rather than cancelling; exact against the official engine on 12 of 12 staged arms. Baton Pass and Shed Tail switch for the first time (`passesState` had been derived and never consumed, so Baton Pass was a no-op turn and Shed Tail paid half its user's HP to stand still). Curse is two moves and the engine had neither. Perish Song counted from 3 instead of 4 and therefore fainted every affected body on both sides a full turn early, on 1,141 corpus uses — the KO itself had always fired. And ROADMAP #81 WIRE 10's measured board regression is one line: the Life Orb toll was being paid by a move

that MISSED. **Two of the five briefed diagnoses were wrong** — the tagger was not testing `selfSwitch === true`, and the substitute doll was not confounded, it was a regression this project introduced at WIRE 7 on a misquoted source line. Census 281/282 → 293/294 live.

THE INSTRUMENT WAS MEASURING ANNOUNCEMENTS, AND THE HEADLINE IS NOW THE BOARD AT THE END OF TURN 1 (3.70.0). `engine/board_state.js` reads HP, status with its counters, items, all seven stat stages, aliveness, every field condition WITH ITS CLOCK and the persistent volatiles out of BOTH engines' live bodies at every turn boundary, after the whole residual phase. Read every figure from `data/state-ladder.json`. **The board at the end of turn 1 is identical in 56.0% of games at the pre-WIRE-1 baseline (1119/1998) and 66.9% at the top rung (1337/1998)**, peaking at 69.3% at WIRE 9; whole-game board agreement went 6.4% -> 15.6% against a protocol number that read 1.8% -> 10.3%, so the wires were real and the protocol number overstated them. **WIRE 10 is a regression the protocol instrument scored as an improvement** — 47 fewer clean turn-1 boards, and diffed per field it is one field, end-of-turn-1 HP wrong in 427 -> 473 games. **41.0% of games whose narration parted inside turn 1 reached an identical board anyway.** The comparator proves itself first: 7 representation mappings red-demonstrated in both directions and 25 planted state divergences, each of which must be caught at the planted boundary and localised to the planted field — 25/25 on all fourteen arms.

What changed (3.69.0): we asked whether making the simulator more correct makes the bot predict better, and the answer is no.

A night of engine fixes made our simulator agree with the official one for much longer before the two part company. The obvious next question is whether that helps — and nobody had checked. So we took the one number the bot actually uses to choose a move (its guess at "how likely am I to win from here"), and we ran it over a large set of real ladder games twice: once through the old engine, once through the fixed one, on the same games with the same dice. The two engines differ in exactly one file, so nothing else can explain a difference.

We are not allowed to tell you what came out, and that is deliberate. The simulator those scores were produced on has not passed its own correctness gate, so every number from this comparison is WITHHELD rather than printed with a warning beside it — this project has learned that a warning beside a number does not stop the number being quoted. There is no score, no error bar, no sample size and no count of who-got-what-right here, and you should not read the silence as "it was small" or "it was big". The moment the gate opens the comparison is re-run and the answer goes back in.

What we CAN say is how it was built, because that part does not depend on the simulator being right: the same games, the same dice, two frozen copies of the code differing in exactly one file, a noise floor measured before any effect was believed, and a re-measurement of the faithfulness a second way to confirm it lines up with itself.

So what IS wrong with the bot's guess? It is wildly overconfident. It uses the full range from 6% to 94%, but reality only moves between 44% and 59%. When it says "94% sure", it wins 59% of the time. That is the thing to fix, and no amount of further engine work touches it.

The honest summary: the engine fixes were real and worth having, and they bought nothing for the decision the bot makes. We are saying so rather than quietly moving on.

What changed (3.68.0): we went back and checked what a night of engine fixes was actually worth, and the honest answer is less than it looked.

One night we found and fixed six things wrong with our engine, and each fix reported a before-and-after number. Then we discovered the before and the after had not been measured on the same set of games, so none of those numbers meant anything. We had kept a frozen copy of the engine at every step, so instead of guessing we replayed all nine of them on one fixed set of 1,995 games.

The headline is a negative one and it is the important one. **The typical game still falls apart after a single turn, exactly as it did before any of the six fixes.** Games that match the official engine all the way through went from 2 to 22 out of 1,995. What did improve is how far into a game we get before the first disagreement — the average roughly doubled — but "further before it breaks" is not the same as "it works", and we should not let the second number stand in for the first.

Two things the replay found that the original before-and-afters had got wrong. **A change nobody wrote up as a fix at all turned out to be worth more than one of the numbered fixes** — it happened to sit just before it, and the pairwise comparison had quietly credited it to its neighbour, which then looked twice as good as it was. And one fix that is definitely correct — an arithmetic rounding error, real and worth fixing — changed **none** of the 1,995 games. Being right and being measurable here are different things, and it is worth saying so out loud rather than rounding it up.

We ran the starting point twice, first and last, with eight replays in between. It gave identical answers, so the differences above are the engine and not the measurement wobbling.

It plays the same real game twice — once in our engine, once in Pokémon Showdown's own — and stops at the first place they disagree. On the first proper run, **159 of 160 games disagreed somewhere**, and most of them within a single turn. That sounds alarming and it is exactly what we wanted: for the first time we can see the gap instead of guessing at it, and every disagreement is a specific, fixable thing rather than a worry.

Two honest caveats we keep attached to that number. It has not yet tested anything past the first turn. And it tested **no mega evolutions at all**, which is about a quarter of what this format actually plays — that is the next job, not a footnote.

The first thing it caught was a claim we had made ourselves a few hours earlier, and it proved us wrong. That is the whole point of building it.

A slide-by-slide, jargon-light tour. The white paper (linked on the last slide) has the math and sources.

Slide 1 — ABRA

A family of small AI models for competitive Pokémon (Champions VGC). Built from thousands of real ladder games. Honest about what it can and can't know.

Slide 2 — The big idea

You **can't reliably predict who wins** a match from the two team sheets — in this format it's close to a coin flip, and even a strong player-rating model barely beats a coin. So ABRA stops trying to call the winner and instead **helps you make better decisions**: what to bring, and what to do turn by turn.

This is the same move sports analytics made with "expected goals" — you can't predict the final score, but you can measure the value of each shot. ABRA is that idea, pointed at Pokémon.

Slide 3 — How it's built

One rule: **keep every real game, and analyse on top of it.** ABRA saves every public ladder replay into a growing library and builds its models on that library — so it gets smarter as more games come in, and never has to re-download anything. It runs on a normal laptop; no special hardware.

Slide 4 — The town of models

Each model is a "house" you can visit on the site:

- **MEDICHAM** — the damage engine. Matches the community standard, but disagrees with the game's OFFICIAL engine by a wide margin, so it is being replaced by that engine and kept only as a lookup.
- **MEW** — plays the official engine against itself, so we are not limited to the games people happen to upload.
- **GURU** — reads the metagame: which team styles beat which, from real results, with error bars.
- **XATU** — reads the opponent: the likely item, ability, and moves behind each Pokémon.
- **PORY** — mid-battle win chance. **Retracted as a finding:** it works out to counting how many Pokémon each side has left and how healthy they are, and it does not beat exactly that baseline.
- **The scoring bot** — the part that finally *looks at the board*.
- **SLOWKING** — the strategist: there's no single best team, so it plays a smart mix.
- **CHOMP** — the team picker: which four to bring, which two to lead.
- **ALAKAZAM** — the coach (in progress): the best move to make, right now.

Slide 5 — The win: the practice opponent can finally see

Everything ABRA claims about "this build beats that build" comes from playing the game out against a practice opponent. That opponent used to pick its move purely by **how popular the move is** — it had no idea what was standing in front of it. So it fired attacks at Pokémon that were flat immune, it used moves that couldn't possibly work, and when there were two enemies to aim at, it picked one by flipping a coin.

We fixed it by **watching what people actually do**. We took about 48,000 real decisions from games where both players' full teams were public — so we could see not just what someone clicked, but everything they *could* have clicked — and learned how much a real player's choice depends on what's in front of them. Nobody wrote the rules; they were measured.

The result, checked on games the learning never saw:

- it now hits the enemy's weakness **half again as often** as before,
- it wastes **a third fewer** moves on things that simply cannot work,
- and it almost never fires at something immune any more.

It is still short of a human on all three, and one thing it still does badly is decide **when to switch** — that's the next job. But the practice opponent is no longer playing blind, which means the numbers it produces are worth more than they were.

PORY also beats a coin flip mid-game, using how many Pokémon each side has left and their health, and it's *calibrated* — when it says 70% it's really about 70%. It's live on the site as a per-turn "you're at X%". **Read the retraction on slide 4 with this, and not after it:** `data/pory-eval.json` ends its own verdict with the sentence *"It does beat a coin … and neither of those is evidence of a learned value function."* Beating a coin is not the bar; PORY ties a model that only counts Pokémon and health, which is the same two things this paragraph credits it with using. The calibration is real and the *learning* is what was withdrawn. (Added 2026-08-22: this slide previously stated the win and the calibration with the retraction two slides away, which reads as a result.)

Slide 6 — The honest parts (this is a feature)

Two things we tested and reported straight, even though they're negatives:

- **Picking the team doesn't beat a coin.** We tested whether the team-picker's choices track who wins — they don't, more than chance. So we don't oversell it. The damage math is still exact and useful.
- **The "rock-paper-scissors" metagame is a hint, not a fact.** The data suggests Trick Room beats Hyper Offense beats Sand beats Trick Room — but each of those rests on only ~13–18 games, so the error bars are wide. We label it suggestive, and it'll firm up as more games arrive.

Added 3.32.0 — three more, and the first one is the worst thing we've found.

- **We thought Rage Powder was deleting attacks. It wasn't — and catching that is the story.** The test we wrote to check it aimed a *Dragon*-type attack at a Pokémon that is immune to Dragon. The redirection worked perfectly and moved the attack onto something that takes zero damage, so the test saw zero either way and we believed the wrong thing for several hours. Re-run against a Pokémon that can actually be hurt, the untouched code works fine. Nothing was broken and nothing we had measured was invalidated. **A test can be wrong in exactly the shape of the bug it is looking for** — that is now the ninth time here, and the first time it fooled us.
- **A real one next door, though.** Lightning Rod and Storm Drain — abilities that pull attacks in — genuinely weren't working, 1,901 uses. Fixed.
- **The number we quote for "how often we disagree with the real game" wasn't repeatable.** It picked its test matchups at random and never wrote down which ones. Two runs on identical code gave 6 and then 3. It now uses a fixed list. *(Corrected 2026-08-22: this used to end "and the honest figure is 4 disagreements in 400". That test has been rebuilt twice since — it is far larger, and it now checks the whole spread of a damage roll rather than only its two ends — so the old score is not comparable and is not restated. The current one is printed by* `node engine/status.js`*.)*
- **The part of the bot that judges "am I winning?" — the figures are WITHHELD as of 2026-08-22.** This bullet used to give the reliability curve in plain words: what the bot wins when it says it is almost certain, and what it wins when it says it is almost lost. Those come from a measurement made by playing games in our own simulator, and that simulator is currently known to be wrong in ways we are still fixing — so every number that came out of it is set aside until it is re-run, rather than printed with a warning label next to it. Two things in this bullet do *not* depend on the numbers and still stand: the earlier reading rested on far too few games to support any claim either way, and the version being measured was not the one the bot actually uses.

Added 3.33.0 — we went looking for the same mistake everywhere else, and found it twice more.

The 3.32.0 note above says a result we'd published as a pass turned out to be unreproducible. Here's *why*, because the reason is more useful than the result: the file we kept recorded the answers and not the settings. It's like keeping a stopwatch time and not writing down which race it was. Two runs with very different settings produced files that were identical, and we could not tell which one we had.

So we checked the other three tests in the same series.

- **"The new search picks a different move much of the time" is a headline with its control missing.** A search that's just guessing disagrees with *everything* — including with itself. So the test measures that too: run the same search twice with different dice and see how often it contradicts itself. That number is the yardstick, and the test printed it to the screen and never saved it. On an early version the yardstick was *higher* than the headline. Believing and having measured are different things, and it's the second one we publish. *(The headline percentage is withheld as of 2026-08-22 — it came out of the*

simulator that is currently being fixed. The point of the bullet is the missing yardstick, which does not need it.)

- **"A single look-ahead costs N milliseconds" was timing something the bot doesn't do.** The measurement quietly used the settings the code falls back to when you don't say, and the bot uses different ones — a randomised playout at triple the length. Not wrong arithmetic; the wrong thing measured. *(The timing itself is withheld as of 2026-08-22, same reason as the two bullets above.)*

Every one of these tests now writes a small companion file recording exactly what it did: how many rollouts, what randomness, what horizon, which version of every file it read, and whether anything was uncommitted at the time. Older results get one reconstructed from the commit that carried them, labelled as a guess rather than a record.

Added 3.34.0 — the clearest example yet of how this goes wrong.

We built a page showing every model, published it, and asked for a review. It was completely broken — one stray quotation mark meant none of it could run, so it showed a title and a blank screen.

Two automated checks looked at that page and both gave it full marks. One counted twelve model cards and found twelve. The other checked that every number on it cited a source, and scored it 100%. Neither of them ever *ran* the page — they read it the way you'd read a document. So a page that could not work passed every test we had.

A person opening the link found it in about two seconds.

The same day, a second version of the same problem: two new pages were written, tested, saved and published to the code repository — and never copied to the folder the live website actually serves. The check meant to catch that compared one specific file, so a *missing* file was invisible to it. Again: found by opening the URL.

Both checks now exist in a form that would have caught them, and both were written the same way — by asking what the check could not see, rather than what it did see.

Saying what we *can't* prove, as plainly as what we can, is the whole point.

We also caught ourselves twice this week, which matters more than any single fix. A result we'd published as a pass — "playing the position out beats just counting what's left" — turned out to be unreproducible from anything we'd kept, and on the evidence that survives it's a tie, not a win. And a claim written in our own notes at 3am about an outside dataset was wrong within four hours; it's corrected in place with the reason, not quietly deleted.

Slide 7 — What's next

The one lesson that changed the plan. We spent a day teaching the bot more about Pokémon — four separate things it didn't know before. All four made no difference at all. Then we changed *what it was trying to do*, twice, and both times it got a lot better.

Teaching it more facts had stopped helping. Changing its goal was what worked.

The reason is simple. The bot was trained to **copy what humans click**. That is a different goal from **winning**. It even shows up in the numbers: the model rates "use a spread move that hits both of them and doesn't hurt my own partner" as a *bad* idea — not because it is bad, but because humans rarely click it. A bot told to avoid its own best plays will avoid them.

- **Teach the two Pokémon to act as a team.** We already built this once. It plays worse, and now we know why: it was taught to *resemble* people rather than to *win*. Retraining it on the right goal is the single

most promising thing on the list.

- **Fix a small unfairness in the training data.** When a Pokémon is locked into one move, we were still showing the bot all four as if it had a choice. It affects about 1 in 15 items.
- **Find out how readable it is.** We have no answer to this. An old figure said a rival bot built to beat ours won 63% of the time; that number is **withdrawn** — it was measured on a third of the features the bot now uses, on a dirty set of games, before most of the engine was fixed. The re-run was ruined when the thing being tested got rebuilt halfway through. So the honest position is that the question is open, not that the answer is bad. A bot can get better on average while staying just as easy to read; those are different questions and only one has been checked.
- **Then, looking ahead.** Right now the bot only thinks about *this* turn. Looking one turn further is a real project, but it's now measured rather than guessed: there are only **72** sensible combinations to consider each turn, not the trillions people assume.
- **ALAKAZAM** — the in-battle coach. A light version runs in your browser; a version that could beat top humans needs a rented cloud computer and months of the AI playing itself.

Something we got wrong, kept here on purpose. For two days nobody updated these documents while the code kept changing. The next person to read them — which was us — then described several models completely incorrectly, including calling one "not built yet" when it had been built, tested, and measured. Documents that lag the work don't just go quiet; they actively mislead.

Slide 8 — You cannot measure something while you are changing it

Four teams work on ABRA at once. That is deliberate — it is why the work was split four ways.

One night all four were running. Their files were kept separate so nobody could overwrite anybody. A 7,100-game experiment — "can anything beat our bot?" — was destroyed anyway.

The reason is simple once you see it. Halfway through the experiment, another team **improved the bot being tested**. So the first half of the experiment played against the old bot and the second half played against the new one. The final number described neither.

Nothing broke. No error appeared. The result looked completely normal, and the automatic checker approved it — because that checker was asking *"is this file newer than the thing it came from?"* and the answer was yes. A file can be newer than something it never actually read.

The fix is not to make the teams take turns. That would throw away the whole reason for having four of them.

Instead, an experiment now works from a **photograph of the bot** rather than the bot itself. Before measuring, we freeze a copy of everything that could change the answer, and the experiment reads the frozen copy. Other teams can rewrite the real thing all night; the experiment never notices.

The honest consequence: **we currently have no answer to "how beatable is our bot?"** The old answer was measured on a version of the bot that no longer exists, and the new attempt is void. Saying so is better than quoting a number we cannot stand behind — that number had been called the most important one in the project.

One thing did survive, and it is genuinely useful: when the bot plays a mirror match against itself it wins 49.7% of the time, which is a coin flip. That confirms our testing setup does not quietly favour one side — a worry an earlier, smaller sample had raised.

Slide 9 — Practising in different conditions from the match

We found something bigger than the bug we went looking for.

In Champions, players often reveal their whole team before the battle. Our bot **uses** that information when it plays. But when we *taught* the bot, we were throwing half of it away — the bot learned without knowing the opponent's abilities or moves, then plays with both.

It affects **half of every decision** it was trained on, and virtually every game. It is like training a driver in fog and then handing them the keys on a clear day: not obviously worse, but they never learned to use what they can now see.

We have not fixed it yet, and that is deliberate. Fixing it means retraining everything, and there is a real question first: sometimes opponents *decline* to reveal their team. A bot trained to depend on information that is sometimes missing can fail badly when it disappears. We have made that mistake before, in exactly this shape, and it is worth one deliberate decision rather than a quick fix.

Update (version 3.40.0). The decision was made: the bot plays with the full team sheet, always, and the rarer no-sheet games are set aside for now. The first half of the retraining is done — the move-picking layer now learns from the same information it plays with, and we *counted* that the information actually arrived during training (99.7% of decisions) instead of assuming it. The second half (the layer that picks pairs of actions) is queued next. No claim yet that the retrained bot is better — that comparison is set up against a frozen copy of the old one and runs next.

Also in this pass: the simulator learned eight more rule families and now covers 181 of the 186 mechanics we probe for, with the remaining 5 named and explained; and we compared our two separate "rulebook" files fact by fact — they disagreed exactly twice, and one of those (how often Iron Head flinches in this format) was a real bug the bot had been playing with. It reads the right number now.

Update (version 3.41.0, same night). The second half of the retraining is done too — the layer that picks pairs of actions now learns from the full team sheet, and we measured what all that sheet information is actually worth by replaying 45,000 real decisions with and without it. Honest answer: the bot judges positions measurably better with the sheet, but we cannot yet show it clicking different moves often enough to prove a win-rate gain — the effect is smaller than our measurement noise. We say it exactly that way rather than rounding up. Meanwhile the simulator's coverage grew again (202 of 205 probed mechanics, 3 remaining, each with a written reason), the mirror-match test hit 100% agreement with the official engine, and two real bugs — one in how Intimidate interacts with certain abilities, one where Sheer Force was hurting its own holder — were found and fixed with proof.

Update (version 3.42.0) — we were teaching the bot moves nobody ever picked.

Will said it first: *"i def dont like just tossing turns because they got outplayed with a move like Encore or Follow Me, these are the basis of VGC."* He was right, and it turned out to be worse than tossing them.

When Encore hits you, the game **takes the controls away** and makes you repeat your last move. The replay just shows you using that move. Because it is a move you legitimately own, every check we had said "looks like a normal choice" and the model dutifully learned it as one. The same thing happened whenever Roar or Whirlwind blew a Pokémon in — the replay writes that arrival exactly like a voluntary switch, so we were teaching the bot to make switches the player never made. **A slice of the recorded actions we were learning from were things no human chose.** The counts are WITHHELD — they come out of the fitted model's own corpus file, which was built on a simulator that has not passed its correctness gate — but the mechanism does not depend on the count: it is not a rounding error being fed to the model, it is outright fiction, and it was concentrated on the turns where the opponent had just outplayed the player. Those are gone now, and counted, so the number comes back the moment it may be quoted.

The other half — Follow Me and Rage Powder — is subtler. The replay only records where an attack *landed*, so when a Pokémon soaks the hit we could not tell which target the player was aiming at. We used to write down "they aimed at the soaker", confidently, which was usually false. Now the model is told the honest thing: it was one of these two, we do not know which, learn accordingly. There is a standard statistical method for exactly this, and we tested it first on data where we had hidden the right answer ourselves — it recovers 97% of the damage when the problem is severe.

And here is the part we are not going to round up. Removing the fiction worked: on those turns the model now puts measurably less confidence in the move nobody picked. The Follow Me fix bought **nothing we can measure** — and our own test had predicted that before we ran it, because those turns are only about 1.3% of the data and the ambiguity is only ever between two targets. Overall accuracy did not move at all. We are doing it anyway, because a wrong label is a wrong label and the model has no way to tell us which of its mistakes came from one.

And then the simulator changed underneath both of those numbers, so we checked them again (3.47.0). A measurement is only worth the version of the code it was taken on. Two of these results had been computed with a simulator that changed the same day, which normally means the results have to be thrown away and re-earned. Before doing that we asked the cheaper question: did the change actually alter any of the 58 things the model looks at? We ran every one of them through the old simulator and the new one, on 1.75 million real decisions, and got byte-for-byte the same answers — so the change was real in the game engine and invisible to the model. We re-ran both measurements anyway, on a corpus that had grown by nearly three hundred games, and every conclusion above came back the same.

One more thing we found and did **not** fix: when a Pokémon is Taunted or Encored its menu shrinks, sometimes to a single legal option, and we were still scoring the player as though they had picked that move out of nine. That is about 1.6% of actions. It is a real problem, it is a different problem, and fixing it means retraining everything again — so it is written down with a number attached rather than quietly bundled in here.

Slide 9b — The rule was there. It was just pointed at everybody. (3.44.0)

Psychic Terrain is a floor that stops fast moves — Fake Out, Sucker Punch, Extreme Speed, the moves that go first. Our simulator knew that. What it did not know is that **the floor only protects Pokémon standing on it.** A Talonflame is in the air. A Pokémon with Levitate is in the air. Fake Out hits them straight through the terrain, and we were blocking it.

Fake Out is the single most-clicked move in this format — nearly thirteen thousand uses in our corpus. So this is not a corner case; it is a rule the practice opponent got wrong in a large slice of the games it plays.

Two things about it are worth more than the fix.

The first: the question *"is this Pokémon on the ground?"* was already answered in three separate places in the same file, by hand, and **the three answers did not agree with each other**. One of them was even keeping a counter of how often it knew it was giving the wrong answer. That counter had been running since the previous release, next to a note saying the information needed to fix it was not available — and the information had arrived a release earlier. There is now one function, and the four places that need the answer all ask it.

The second: our own test suite could not have caught this. The existing check for Psychic Terrain aims the blocked move at a **Garchomp**, which is standing on the ground, so it passes whether the rule is right or wrong. Every instrument we own asks *"does this mechanic happen?"*. None of them asked *"does it happen*

only where it should?" — and that is the fourth bug of exactly this shape in two days. The new check has five arms, and one of them exists purely to catch a mistake we nearly made: Orthworm's Earth Eater makes it immune to Ground attacks, which looks in our data exactly like Levitate, and Orthworm is very much on the floor. We asked the official game engine and it told us so.

Slide 9b — We said we had 97 bugs. We had none of them.

A tool of ours changes a fact the simulator is supposed to read, then watches whether the simulator's behaviour moves. If nothing moves, the simulator was not really reading that fact. It flagged **97** of these, and they were passed on as 97 bugs.

Will asked what the top two actually were. **Both were fine.** Life Orb's damage boost *is* read from the data; only its recoil is written against the item's name, which is untidy but not wrong today. Light Screen's number is ignored **deliberately** — the data file carries the singles-format value, and this is a doubles engine where the real number is different. Using the data file's number would have made every Light Screen look a third better than it is.

The tool could see that nothing moved. It could not see **why**. It now answers that too, by reading the simulator's own source code rather than anybody's comments about it, and sorts every finding into four boxes: nothing implements this at all (the real bugs), the engine substitutes its own value on purpose, the engine works off the name instead of the data, and the test simply never reached that branch. **None of the 97 is in the first box.** The count we now track is that first box only — 163 items, none of them from the 97 — because a number that includes false alarms is a number people stop reading.

The rule was checked against three cases we had already worked out by hand, and it refuses to publish anything if it cannot reproduce all three.

Slide 9c — Everything that changes accuracy was switched off, and one move was charged for and never delivered (3.56.0)

Some moves miss. Some things make you miss less: Coil sharpens your aim, a Wide Lens is an item that does the same. Some things make you miss more: Sand Veil hides a Pokémon in a sandstorm. And No Guard means nobody ever misses at all. All four are in the game we are modelling, and together they show up about five thousand times in our replay store.

We measured each of them the boring way — play it out with the thing, play it out without the thing, compare. **All four returned exactly the same number both ways.** Not close, not within noise: identical, to the digit. Something that changes nothing at all is not a subtle bug. Every one of them was simply not connected.

Three separate reasons, which is the part worth noticing. A translation table turned the words "accuracy" and "evasion" into nothing, so a move that was supposed to raise three of your stats quietly raised two. Items and abilities were never consulted for accuracy anywhere. And the function that decided whether you hit **was not given the attacker or the target** — it was handed a move name and asked to answer a question that depends on who is holding what. It could not have got that right even in principle.

In the same pass: **Substitute took a quarter of your health and gave you nothing.** The doll was paid for and never built. That is 1,976 clicks in the store of a move that was strictly worse than doing nothing — the simulator would tell you the cost and then hand you an empty box.

The tempting fix for Substitute was the bigger bug, and we measured it before writing it. Some moves go *through* a Substitute — sound-based ones do, famously. It would have been natural to write the rule as "sound gets through". We checked: the three moves that most often need to get through in this format are Encore,

Taunt and Disable, together about seven thousand uses, and **not one of them is a sound move.** A tidy rule would have walled all three.

Coverage is now 231 of 232 probed mechanics. The one that remains has a written reason, as do five more rules we know are absent — the largest of those needs information the damage function is never handed, so wiring it is a design decision rather than a fix, and it is written down as one instead of being quietly patched.

Slide 9d — The simulator could not say what it did, only where it ended up (3.58.0)

The problem. We check our simulator against the official one by comparing *results* — how much damage, what the board looked like after a turn. When they disagree, that tells you *that* something is wrong and almost nothing about *what*.

What the official simulator has that we did not. A running commentary. Every decision it makes it writes down, labelled: *Incineroar used Fake Out. Garchomp's Attack fell. The Focus Sash broke.* It is a transcript with the reason attached to every line.

What changed. Our simulator writes the same transcript now, in the same format, off by default so it costs nothing when nobody asks. The list of things it can write down was **read out of the official simulator's source code**, not typed from memory — and it checks itself two ways: it fails if we claim to write something the real one never writes, and it fails if the real one writes something we neither write nor have an explanation for. Ninety-one kinds of event exist; we write thirty-six, and the other fifty-eight each have a sentence saying why not.

What it caught on the first night. Two things, and both are about our own measuring tools rather than about the game:

- Our damage test only ever compares the *lowest* and *highest* possible rolls. It never looked at the fourteen in between — and in between, the two simulators do not agree about how likely each one is. We had been reporting 149 out of 150 correct on a comparison that could not see the middle.
- Our simulator does things in a different *order* inside a single attack than the real one does. The board ends up identical, which is exactly why the old test said everything was fine.

Neither is fixed yet, on purpose. Changing how a damage roll is picked would move every measurement this project has ever recorded. The point of tonight was to build the instrument that can see it.

Slide 9e — Somebody built a bot that beats a pro. Any strong human learns to beat it in five games. (3.62.2)

The result. A university team (Angliss, Cui, Hu, Rahman and Stone, published at AAMAS 2026) built a Pokémon doubles bot the proper modern way: show it 700,000 human games so it learns to imitate people, then let it play itself millions of times to get better. It works. **In a mirror match it beat a World Championships competitor.**

And then they measured how easy it is to beat, and the answer was: completely. They trained a second bot whose only job was to beat the first one, and it did — almost every time. Their human tester said it plainly: *the agent is strong at first, but after enough games in a row, strong players adapt and beat it.* Against a merely *advanced* player it won two games out of five.

We think that is the whole problem, not a bug in their work. Here is the thing to picture. Their bot is a **memorised answer sheet**: for every situation, one answer, worked out in advance and then frozen. That is a great way to play chess, where both players can see everything. It is a terrible way to play poker, where they can't — because once your opponent notices you always fold in a spot, you lose that spot forever. **Pokémon is poker, not chess.** You cannot see which four of their six they brought, what they're holding, or their last move. Poker figured this out between 2007 and 2021, and the answer was: **don't memorise, re-think every hand.**

So our headline number changes. We used to ask "how often do we win?" We now ask **"how easily can a prepared opponent read us?"** That is a harder, more honest question, and it is the one their number answers too — so the two projects can be compared even though the bots can never actually play each other. Their score is about 100% readable. **Ours is currently unknown, and saying so is the point** — we've made our headline a number we can't produce yet rather than a number that flatters us.

Our bet — and it is a bet, not a claim. A bot that *recomputes* from scratch every turn should be harder to read than one that *recalls*. There is nothing to memorise about it. Whether that survives in a game where both players move at once, dice are rolled, and the whole thing is over in six turns, **nobody knows. That's the experiment.**

What we do if we're wrong. We use their recipe. It's published, it's open source, and it works. Being wrong here would be a real finding about Pokémon, not a failure.

Slide 9f — We had a number that justified two years of work, and it was wrong by five times (3.62.2)

The claim. We wrote our own Pokémon simulator instead of using the official one, and the reason written down in every document was: **the official one is 117 times slower.** That number decided the architecture.

We measured it again. It's about 25 times slower, not 117.

Why we're telling you rather than quietly fixing it. Three things.

1. The old number stays in the documents, with a dated correction beside it, because this project does not rewrite a past conclusion as though it were never made.
2. **The decision it justified is still right** — 25 times slower is still far too slow to think during a live game. So nothing is being rebuilt.
3. **But the reason has to be a real reason.** "117 times" was doing work it hadn't earned. The honest version is now a claim you could prove us wrong about: *we wrote a fast simulator so the bot can re-think every turn — so the simulator is worth building* ***if and only if*** *re-thinking actually helps.* We are about to run the test that decides it.

The part that should worry a reader most. We have three different speed measurements of our own simulator, taken two weeks apart, that disagree by a factor of ten — and **not one test noticed.** We measure win rates to three decimal places and check them against a noise floor. We had never once checked the speed of the thing everything else runs on.

Slide 10 — Read more

Full technical detail, math, and sources: **ABRA white paper**. Also: **project summary** · **technical docs** · **model ledger**.